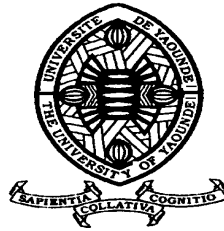


UNIVERSITÉ DE YAOUNDE I

ÉCOLE NATIONALE SUPÉRIEURE
POLYTECHNIQUE DE YAOUNDE

DEPARTEMENT DE GENIE
INFORMATIQUE



UNIVERSITY OF YAOUNDE I

NATIONAL ADVANCED SCHOOL
OF ENGINEERING OF YAOUNDE

DEPARTMENT OF COMPUTER
ENGINEERING

CONCEPTION D'UNE ARCHITECTURE MLOps Cloud SUR AWS

Mémoire de fin d'études

Présenté et soutenu par

TAMBOU DJOMOU FRANCK DONALD

Matricule : 23P732

En vue de l'obtention du :

**DIPLÔME D'INGÉNIEUR DE CONCEPTION EN ARTS NUMÉRIQUES
OPTION : ARTS ET INTELLIGENCE ARTIFICIELLE**

Sous la supervision de :

Pr. Thomas BOUETOU BOUETOU, Professeur, UYI/ENSPY

Dr. Austin WAFFO, Chargé de cours , UYI/ENSPY

Ing. Junior TEMGOUA , CTO , DESEMITS LLC

Devant le jury composé de :

Président : **Pr. X**, Maître de conférence, UYI/ENSPY

Examineur : **Dr. X**, Chargé de cours, UYI/ENSPY

Rapporteur 1 : **Pr. Thomas BOUETOU BOUETOU**, Professeur, UYI/ENSPY

Rapporteur 2 : **Dr. Austin WAFFO**, Chargé de cours , UYI/ENSPY

Invité : **Mme Fridoline MBIA** , RH , TECTRA

Année académique 2025-2026

Mémoire soutenu en Juillet 2026



CONCEPTION D'UNE ARCHITECTURE MLOPS CLOUD SUR AWS

Mémoire de fin d'études

Présenté et soutenu par

TAMBOU DJOMOU FRANCK DONALD

Matricule : 23P732

En vue de l'obtention du :

**DIPLÔME D'INGÉNIEUR DE CONCEPTION EN ARTS NUMÉRIQUES
OPTION : ARTS ET INTELLIGENCE ARTIFICIELLE**

Année académique 2025-2026

Mémoire soutenu en Juillet 2026

DÉDICACE



À Mes Parents



REMERCIEMENTS

La réalisation de ce mémoire n'aurait pas été possible sans le soutien et les conseils de nombreuses personnes. Nous leur adressons notre sincère gratitude, en particulier à :

- ☛ **Dieu Tout-Puissant**, pour la vie, la santé, la force, la sagesse et les nombreuses grâces qu'Il nous a accordées tout au long de ce parcours.
- ☛ **Pr. X**, qui nous fait l'honneur de présider ce jury.
- ☛ **Pr. X**, qui a accepté d'examiner ce travail et dont les observations et recommandations contribueront à son amélioration.
- ☛ **Pr. Thomas BOUETOU** et **Dr. Austin WAFFO KOUHOUE**, qui ont accepté d'être les rapporteurs de ce mémoire et pour l'intérêt qu'ils portent à ce travail.
- ☛ **Ing. Junior TEMGOUA**, notre encadreur professionnel et *Chief Technology Officer* de **DESEMITS LLC**, pour sa disponibilité, son accompagnement, son expertise technique ainsi que les nombreuses connaissances professionnelles qu'il nous a transmises durant ce stage.
- ☛ **Prof. Raoul Domingo AYISSI**, Directeur de l'École Nationale Supérieure Polytechnique de Yaoundé, pour la qualité de la formation dispensée au sein de cette prestigieuse institution.
- ☛ **Pr. Thomas BOUETOU**, Chef du Département de Génie Informatique de l'École Nationale Supérieure Polytechnique de Yaoundé, pour son engagement constant dans la formation des futurs ingénieurs.
- ☛ L'ensemble des responsables et collaborateurs de **DESEMITS LLC**, pour leur accueil, leur confiance et le cadre professionnel enrichissant qui nous a permis de mener à bien ce projet de fin d'études.
- ☛ Tout le corps enseignant de l'École Nationale Supérieure Polytechnique de Yaoundé, pour la qualité des enseignements dispensés et les compétences acquises durant notre formation d'ingénieur.
- ☛ Nos parents, ainsi que toute notre famille, pour leur amour, leurs sacrifices, leurs prières, leurs encouragements et leur soutien indéfectible tout au long de notre parcours.



👉 Nos camarades de promotion, nos amis ainsi que toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce mémoire par leurs conseils, leur soutien ou leurs encouragements.

Enfin, nous adressons nos sincères remerciements à toutes les personnes qui, directement ou indirectement, ont contribué à l'aboutissement de ce travail et que nous n'avons pu citer individuellement.



RÉSUMÉ

L'industrialisation des modèles d'intelligence artificielle représente aujourd'hui un enjeu majeur pour les entreprises souhaitant déployer des solutions performantes en environnement de production. Si les modèles développés en phase expérimentale offrent généralement de bons résultats, leur passage à l'échelle est souvent limité par le volume croissant des données, les besoins en ressources de calcul et les contraintes liées à l'automatisation des traitements. Ce mémoire, réalisé au sein de **DESEMITS LLC**, porte sur la conception d'une architecture Cloud orientée MLOps destinée à accompagner le passage à l'échelle d'un modèle de vision par ordinateur utilisé pour la détection d'anomalies sur des cultures agricoles. L'objectif est de concevoir une infrastructure distribuée capable d'assurer un traitement efficace de grands volumes d'images tout en garantissant la scalabilité, la disponibilité et l'optimisation des coûts. La solution proposée repose sur les services **Amazon Web Services (AWS)**, notamment Amazon S3 pour le stockage distribué des données, Amazon EMR pour le calcul distribué et Apache Spark (PySpark) pour le traitement parallèle des images. Cette architecture permet de séparer les ressources de stockage et de calcul, d'améliorer les performances du pipeline de traitement et de faciliter son intégration dans une démarche MLOps. Les expérimentations réalisées montrent que l'architecture proposée répond efficacement aux exigences de montée en charge en offrant une solution modulaire, évolutive et adaptée au traitement massif des données. Elle constitue ainsi une base robuste pour le déploiement et l'industrialisation de modèles de vision par ordinateur dans un environnement Cloud.

Mots-clés : **Intelligence artificielle, MLOps, Cloud Computing, Amazon Web Services, Amazon EMR, Amazon S3, Apache Spark, Calcul distribué, Vision par ordinateur, Scalabilité.**

ABSTRACT

The industrialization of artificial intelligence models has become a major challenge for organizations seeking to deploy reliable and efficient AI solutions in production environments. Although models developed during the experimental phase generally achieve high performance, their scalability is often constrained by the increasing volume of data, computational resource requirements, and the need for automated processing pipelines. This thesis, carried out at **DESEMITS LLC**, focuses on the design of a Cloud-based MLOps architecture to support the scaling of a computer vision model used for crop anomaly detection. The main objective is to design a distributed infrastructure capable of efficiently processing large volumes of images while ensuring scalability, high availability, and cost optimization. The proposed solution relies on **Amazon Web Services (AWS)**, using Amazon S3 for distributed data storage, Amazon EMR for distributed computing, and Apache Spark (PySpark) for parallel image processing. This architecture separates storage and computing resources, improves the performance of the processing pipeline, and facilitates its integration into an MLOps workflow. Experimental results demonstrate that the proposed architecture effectively addresses scalability requirements by providing a modular, flexible, and scalable solution for large-scale data processing. It therefore establishes a solid foundation for the deployment and industrialization of computer vision models in a Cloud environment.

Keywords : **Artificial Intelligence, MLOps, Cloud Computing, Amazon Web Services, Amazon EMR, Amazon S3, Apache Spark, Distributed Computing, Computer Vision, Scalability.**

TABLE DES MATIÈRES

Dédicace	i
Remerciements	ii
Résumé	iv
Abstract	v
Table des matières	vi
Liste des figures	viii
Liste des tableaux	ix
Liste des abréviations et sigles	x
Glossaire	xii
Introduction générale	1
1 Généralités et état de l'art	4
1.1 Généralités	5
1.1.1 Le MLOps (Machine Learning Operations)	5
1.1.2 Le Cloud Computing : Catalyseur de l'IA à grande échelle	7
1.2 État de l'art	10
1.2.1 Cas d'étude 1 : Netflix – Architecture MLOps de référence, orchestration hybride et passage à l'échelle	10
1.2.2 Cas d'étude 2 : Capital One – MLOps ultra-sécurisé, conformité et gouver- nance dans le secteur bancaire (FSI)	13
1.2.3 Cas d'étude 3 : Riot Games – Inférence et déploiement mondialisé à très faible latence (Edge Computing)	16
1.2.4 Cas d'étude 4 : Expedia Group – Standardisation MLOps, conteneurisation et inférence multi-région	18

1.2.5	Cas d'étude 5 : Plateforme MLOps sécurisée basée sur Terraform, GitHub et Amazon SageMaker	20
1.2.6	Bilan de l'état de l'art et Positionnement technologique du projet	22
2	Méthodologie de conception	25
2.1	Planification du projet	26
2.2	Analyse du problème et définition des besoins	27
2.2.1	Analyse du problème	27
2.2.2	Besoins fonctionnels	27
2.2.3	Besoins non fonctionnels	29
2.2.4	Acteurs du système	31
2.2.5	Diagramme de cas d'utilisation	32
2.3	Conception de la solution	33
2.3.1	Architecture logique	33
2.3.2	Architecture physique	34
2.4	Résumé du chapitre	36
3	Implémentation et résultats	38
3.1	Implémentation matérielle	39
3.1.1	Configuration de l'environnement Cloud	39
3.1.2	Ressources d'infrastructure nécessaires (Services Cloud)	39
3.1.3	Bilan FinOps et Consommation des ressources	42
3.2	Implémentation logicielle	43
3.2.1	Outils logiciels et environnement de travail collaboratif	43
3.2.2	Choix des technologies	44
3.2.3	Développement des modules	44
3.3	Résultats	45
3.3.1	Intégration et tests	45
3.3.2	Protocole Expérimental	45
3.3.3	Résultats Expérimentaux	46
3.4	Discussion	46
3.5	Résumé du chapitre	47
	Conclusion Générale	48
	Références bibliographiques	50



LISTE DES FIGURES

2.1	Modélisation fonctionnelle du flux de données vers le Data Lake	27
2.2	Flux fonctionnel d'Intégration et de Déploiement Continu (CI/CD)	28
2.3	Diagramme de cas d'utilisation du système hybride (Data Engineering & MLOps) . .	33
2.4	Architecture logique du pipeline de traitement distribué et d'orchestration MLOps . .	34
2.5	Architecture physique détaillée de la solution MLOps sur Amazon Web Services . . .	36
3.1	Virtual Private Cloud (VPC) créé pour l'isolation du projet	40
3.2	Organisation du Data Lake dans le bucket Amazon S3	41
3.3	Déploiement du cluster Amazon EMR (1 Nœud maître et 4 Workers)	41
3.4	Déploiement du cluster Amazon EMR (1 Nœud maître et 4 Workers) : Fin	42
3.5	Connexion sécurisée au nœud maître Amazon EMR via SSH	42
3.6	Interface d'Amazon EMR Studio	43
3.7	Workspace collaboratif associé à un cluster Amazon EMR	44
3.8	Interface Apache Spark présentant la répartition des tâches sur le cluster	45

LISTE DES TABLEAUX

1.1	Cartographie de l'Architecture de Référence MLOps (Netflix vs AWS)	13
1.2	Cartographie de l'Architecture MLOps Sécurisée (Secteur Bancaire / FSI)	16
1.3	Cartographie de l'Architecture Cloud à très faible latence (Riot Games / AWS)	18
1.4	Cartographie de l'Architecture et des Services Cloud (Expedia Group)	20
1.5	Cartographie de l'Architecture de Référence (AWS, Terraform, GitHub)	22
2.1	Exigences fonctionnelles liées au prétraitement des images agricoles	28
2.2	Exigences fonctionnelles liées à la Plateforme MLOps	29
2.3	Matrice de synthèse des besoins non fonctionnels de l'architecture	31
2.4	Cartographie des acteurs et de leurs responsabilités dans le système	32
3.1	Environnement matériel et logiciel de l'implémentation	39
3.2	Organisation du Data Lake sur Amazon S3	40
3.3	Bilan de l'évaluation de l'infrastructure Cloud	46

LISTE DES ABRÉVIATIONS ET SIGLES

- **ACP** : Analyse en Composantes Principales
- **API** : Application Programming Interface (Interface de programmation d'application)
- **AWS** : Amazon Web Services
- **CACE** : Changing Anything Changes Everything (Modifier une chose modifie tout)
- **CD** : Continuous Deployment / Delivery (Déploiement / Livraison continu)
- **CI** : Continuous Integration (Intégration continue)
- **CNN** : Convolutional Neural Network (Réseau de Neurones Convolutionnels)
- **CPU** : Central Processing Unit (Unité centrale de traitement)
- **CT** : Continuous Training (Entraînement continu)
- **DevOps** : Development and Operations (Développement et Opérations)
- **DL** : Deep Learning (Apprentissage profond)
- **EC2** : Elastic Compute Cloud
- **EMR** : Elastic MapReduce
- **FinOps** : Financial Operations (Gestion financière des ressources Cloud)
- **GCP** : Google Cloud Platform
- **GPU** : Graphics Processing Unit (Processeur graphique)
- **HTTP** : Hypertext Transfer Protocol
- **IA** : Intelligence Artificielle
- **IaaS** : Infrastructure as a Service (Infrastructure en tant que service)
- **IAM** : Identity and Access Management (Gestion des identités et des accès)
- **JSON** : JavaScript Object Notation
- **ML** : Machine Learning (Apprentissage automatique)
- **MLOps** : Machine Learning Operations (Opérations d'apprentissage automatique)
- **PaaS** : Platform as a Service (Plateforme en tant que service)
- **PCA** : Principal Component Analysis (Équivalent anglais de l'ACP)
- **PoC** : Proof of Concept (Preuve de concept / Expérimentation)



- **RAM** : Random Access Memory (Mémoire vive)
- **REST** : Representational State Transfer
- **RGPD** : Règlement Général sur la Protection des Données
- **S3** : Simple Storage Service (Service de stockage d'Amazon)
- **SaaS** : Software as a Service (Logiciel en tant que service)
- **SLA** : Service Level Agreement (Accord de niveau de service)
- **SLI** : Service Level Indicator (Indicateur de niveau de service)
- **SLO** : Service Level Objective (Objectif de niveau de service)
- **SRE** : Site Reliability Engineering (Ingénierie de la fiabilité des sites)
- **UML** : Unified Modeling Language (Langage de modélisation unifié)
- **VPC** : Virtual Private Cloud (Réseau privé virtuel)



GLOSSAIRE

- **Cluster (Grappe de serveurs) :** Ensemble d'ordinateurs virtuels ou physiques (nœuds) interconnectés qui travaillent de concert pour exécuter des tâches de calcul distribué, comme le permet le service Amazon EMR [1, 2].
- **Data Lake (Lac de données) :** Référentiel centralisé (tel qu'Amazon S3) permettant de stocker d'immenses volumes de données brutes, structurées ou non structurées, à n'importe quelle échelle et de manière hautement durable [3, 4].
- **Dérive des données (Data Drift) :** Phénomène par lequel les propriétés statistiques des données en production changent au fil du temps par rapport aux données d'entraînement, entraînant une dégradation des performances du modèle [5, 6].
- **Hyperparamètre :** Paramètre dont la valeur est définie avant le début du processus d'apprentissage d'un modèle et qui contrôle la manière dont l'algorithme apprend [6, 7].
- **Inférence :** Phase de production durant laquelle un modèle d'apprentissage automatique préalablement entraîné est utilisé pour analyser de nouvelles données et générer des prédictions [8, 9].
- **Instance Spot :** Capacité de calcul informatique excédentaire proposée par un fournisseur Cloud (comme AWS) à un tarif très réduit par rapport au prix classique, couramment utilisée pour l'optimisation FinOps [7, 10].
- **Microservices :** Architecture logicielle consistant à structurer un système complexe (comme un pipeline MLOps) en un ensemble de petits services indépendants, déployables séparément et faiblement couplés [11, 12].
- **Nœud (Node) :** Une machine individuelle (serveur virtuel) faisant partie d'un cluster de calcul. On distingue souvent le nœud maître (qui orchestre) des nœuds travailleurs (qui exécutent les calculs en parallèle) [1, 2].
- **Orchestrateur :** Outil logiciel (comme Apache Airflow ou AWS Step Functions) permettant d'automatiser, de planifier et de coordonner l'exécution de flux de travaux complexes de manière séquentielle ou parallèle [13, 14].
- **Passage à l'échelle (Scalabilité) :** Capacité d'une architecture informatique à s'adapter dynamiquement à une augmentation massive de la charge de travail (volume de données) sans perte de performances [10, 15].



- **Pipeline** : Chaîne de traitement informatique automatisée où la sortie d'une étape sert directement d'entrée à l'étape suivante, structurant ainsi le flux de préparation des données et d'apprentissage [16, 17].
- **Preuve de Concept (PoC)** : Réalisation expérimentale (souvent sur une machine locale) d'un modèle d'IA visant uniquement à démontrer sa faisabilité théorique ou mathématique avant son industrialisation [18, 19].



INTRODUCTION GÉNÉRALE

CONTEXTE

L'intelligence artificielle (IA) et plus particulièrement l'apprentissage automatique (Machine Learning) connaissent aujourd'hui un essor fulgurant, transformant en profondeur la manière dont les organisations exploitent leurs données. Cependant, si le développement de modèles d'IA performants en laboratoire est devenu monnaie courante, leur passage à l'échelle et leur déploiement en environnement de production restent des défis technologiques majeurs. En effet, les modèles déployés sont confrontés à des volumes de données massifs et à une dégradation inévitable de leurs performances dans le temps, un phénomène connu sous le nom de dérive des données [20]. Pour surmonter ces obstacles, l'industrie s'oriente massivement vers le Cloud Computing, qui offre une puissance de calcul et de stockage à la demande, et vers les pratiques du MLOps (*Machine Learning Operations*). Le MLOps unifie le développement des systèmes d'apprentissage automatique et leur exploitation, permettant d'automatiser le cycle de vie des modèles, de leur entraînement initial jusqu'à leur supervision et leur réentraînement continu [20]. C'est dans ce contexte d'industrialisation que s'inscrit le besoin de concevoir des architectures robustes, capables de soutenir des pipelines de données complexes tout en garantissant la fiabilité et la reproductibilité des déploiements.

PROBLÉMATIQUE

Si le Cloud Computing et le MLOps offrent des outils puissants pour industrialiser l'intelligence artificielle, leur application concrète soulève des problématiques d'ingénierie critiques. Les architectures traditionnelles ou locales saturent rapidement face à l'augmentation des flux de données. De plus, un modèle figé perd inévitablement en précision face à l'évolution des données du monde réel. L'absence d'une boucle de rétroaction permettant l'apprentissage continu entraîne souvent une maintenance complexe, un déclin des prédictions et une explosion des coûts opérationnels [20].

Dans ce contexte, une question centrale émerge :



Problématique centrale

Comment concevoir une architecture MLOps Cloud sur AWS qui permette la continuité d'apprentissage d'un modèle et de garantir son passage à l'échelle face à des données massives, tout en optimisant les performances et les coûts d'exploitation ?

OBJECTIFS DU MEMOIRE

L'objectif principal de ce mémoire est de concevoir et d'implémenter une architecture Cloud MLOps sur AWS permettant l'industrialisation et l'apprentissage continu d'un modèle d'intelligence artificielle. Cela passera par les objectifs spécifiques suivants :

- Analyser et comparer les principales plateformes Cloud et approches MLOps existantes afin de justifier les choix technologiques retenus.
- Concevoir une architecture distribuée découplant strictement les ressources de stockage et les ressources de calcul pour garantir un passage à l'échelle optimal.
- Modéliser un pipeline automatisé intégrant l'ingestion de nouvelles données, la continuité d'apprentissage (réentraînement) et l'inférence du modèle.
- Déployer la solution sur AWS et évaluer ses performances techniques et financières (FinOps).

MÉTHODOLOGIE ET STRUCTURE DU MEMOIRE

Pour atteindre ces objectifs, nous avons suivi une approche méthodologique qui est la suivante :

- Effectuer une revue de la littérature afin de comprendre l'existant, de définir les concepts du Cloud et du MLOps, et de mieux appréhender les contours de notre projet ;
- Réaliser l'analyse des besoins fonctionnels et non fonctionnels pour définir le périmètre du système ;
- Effectuer une modélisation théorique de l'architecture logique et de l'architecture physique de la solution ;
- Implémenter la solution sur l'infrastructure AWS et élaborer un protocole d'expérimentation pour valider les résultats.

Le mémoire sera structuré comme suit :

👉 Chapitre 1 : Généralités et état de l'art

Dans ce chapitre, nous présentons dans un premier temps des généralités structurées





autour de deux axes : le MLOps (son importance, sa méthodologie, ses défis et son cycle de vie) et le Cloud Computing (ses fonctions et ses limites juridiques et sécuritaires). Dans un second temps, nous dressons un état de l'art comparatif des grandes plateformes Cloud et des architectures MLOps existantes. Pour finir, nous faisons un bilan afin de positionner notre démarche.

👉 **Chapitre 2 : Méthodologie**

Dans ce chapitre, nous présentons en détail la méthodologie adoptée pour la réalisation du projet. Nous exposons la méthodologie de développement, suivie d'une analyse approfondie du problème (besoins, acteurs, cas d'utilisation). Enfin, nous détaillons la conception de la solution à travers ses architectures logique et physique.

👉 **Chapitre 3 : Implémentation et résultats**

Dans ce dernier chapitre, nous présentons l'environnement d'implémentation et le déploiement de l'infrastructure sur AWS. Nous discutons ensuite des résultats expérimentaux obtenus, en analysant les performances de l'architecture, sa capacité de passage à l'échelle et son optimisation des coûts.

Nous terminons par une conclusion générale, qui présente le bilan de ce travail, ses limites et des propositions de perspectives pour les recherches futures.



GÉNÉRALITÉS ET ÉTAT DE L'ART

Ce chapitre s'articule autour de trois axes principaux : les généralités (sur le MLOps et le Cloud Computing) dans un premier temps, puis l'exploration des architectures MLOps de référence déployées dans l'industrie dans un second temps. Nous finissons par un bilan global afin de positionner et de justifier nos choix technologiques pour la suite de ce mémoire.

Sommaire

1.1	Généralités	5
1.1.1	Le MLOps (Machine Learning Operations)	5
1.1.2	Le Cloud Computing : Catalyseur de l'IA à grande échelle	7
1.2	État de l'art	10
1.2.1	Cas d'étude 1 : Netflix – Architecture MLOps de référence, orchestration hybride et passage à l'échelle	10
1.2.2	Cas d'étude 2 : Capital One – MLOps ultra-sécurisé, conformité et gouvernance dans le secteur bancaire (FSI)	13
1.2.3	Cas d'étude 3 : Riot Games – Inférence et déploiement mondialisé à très faible latence (Edge Computing)	16
1.2.4	Cas d'étude 4 : Expedia Group – Standardisation MLOps, conteneurisation et inférence multi-région	18
1.2.5	Cas d'étude 5 : Plateforme MLOps sécurisée basée sur Terraform, GitHub et Amazon SageMaker	20
1.2.6	Bilan de l'état de l'art et Positionnement technologique du projet	22



1.1 Généralités

Cette section pose le cadre conceptuel et théorique indispensable à notre étude. L'industrialisation d'un système intelligent ne repose pas uniquement sur la qualité de son algorithme, mais sur une méthodologie et une infrastructure capables de le soutenir à grande échelle. Cette partie s'articule autour de deux piliers technologiques fondamentaux : le **MLOps** (*Machine Learning Operations*) et le **Cloud Computing**.

1.1.1 Le MLOps (Machine Learning Operations)

1.1.1.1 Du DevOps au MLOps : Définition et nécessité

Le MLOps tire ses origines des pratiques d'ingénierie logicielle traditionnelles, particulièrement du DevOps (*Development and Operations*). Le DevOps a révolutionné la création de logiciels en fusionnant le développement et l'exploitation pour introduire l'Intégration Continue (CI) et la Livraison Continue (CD), garantissant des déploiements rapides et fiables [21].

Cependant, l'application stricte du DevOps aux systèmes d'intelligence artificielle s'est révélée insuffisante. Contrairement aux logiciels traditionnels dont le comportement est dicté par des règles de code statiques, les systèmes d'apprentissage automatique déduisent leur comportement à partir des données [21]. Le MLOps se définit donc comme une discipline d'ingénierie à l'intersection de l'apprentissage automatique, de l'ingénierie des données (*Data Engineering*) et du DevOps [21].

La différence fondamentale réside dans l'évolution du système en production. Le DevOps automatise le cycle de vie d'un seul artefact (le code), tandis que le MLOps doit gérer un cycle complexe impliquant trois artefacts interdépendants : le code, les données et le modèle [21, 22]. Lorsqu'un modèle est confronté aux données changeantes du monde réel, ses performances se dégradent inévitablement. Le MLOps introduit ainsi une nouvelle pratique : l'**Entraînement Continu** (*Continuous Training* - CT), permettant de réentraîner le modèle pour qu'il reste pertinent au fil du temps [22, 23].

1.1.1.2 Les défis de l'IA et la « dette technique cachée »

Développer un modèle prédictif performant en laboratoire est aujourd'hui rapide et peu coûteux, mais son maintien en production est complexe. Comme le soulignent Sculley *et al.* dans leurs travaux fondateurs chez Google, le code spécifique à l'apprentissage automatique ne représente qu'une infime fraction d'un système IA en exploitation [24]. L'essentiel du système est constitué d'une vaste infrastructure (souvent appelée « code de liaison » ou *glue code*) dédiée à la collecte des données, à la vérification, et à la gestion des ressources [24].

L'absence d'une architecture MLOps rigoureuse conduit à l'accumulation d'une « **dette technique cachée** » (*Hidden Technical Debt*) [24]. Les systèmes ML sont vulnérables à des défis systémiques uniques :





- **L'enchevêtrement (Principe CACE) :** Dans un modèle ML, les signaux d'entrée ne sont jamais réellement indépendants. Le principe CACE (*Changing Anything Changes Everything*) stipule que la modification d'un seul hyperparamètre, ou l'ajout d'une seule caractéristique, modifie les poids de toutes les autres et altère le comportement de tout le système [24].
- **Les dépendances de données instables :** Pour fonctionner, le modèle consomme des signaux d'entrée. Si la distribution statistique de ces données change silencieusement dans le monde réel (un phénomène appelé dérive des données ou *Data Drift*), les prédictions se dégradent sans qu'aucune erreur logicielle ne soit déclenchée [24].
- **Les boucles de rétroaction cachées :** Un modèle déployé finit souvent par influencer les futures données qu'il va ingérer (par exemple, un système de recommandation qui modifie le comportement de clic des utilisateurs), créant des boucles de rétroaction extrêmement complexes à diagnostiquer [24].

1.1.1.3 Principes fondamentaux et cycle de vie MLOps

Pour rembourser cette dette technique et automatiser le déploiement, l'architecture MLOps s'appuie sur des modèles de conception (*design patterns*) rigoureux. Selon la littérature scientifique [21] et les standards de l'industrie (CD4ML) [23], un système MLOps complet repose sur les principes suivants :

- **L'orchestration des flux de travail (Pipelines) :** Les tâches complexes (ingestion des données, ingénierie des caractéristiques, entraînement du modèle) sont coordonnées par des graphes orientés acycliques (DAG). Cette automatisation garantit que chaque étape est exécutée dans le bon ordre de manière répétable [21, 23].
- **Le versionnage unifié et la Reproductibilité :** Pour garantir une reproductibilité totale, le MLOps exige le versionnage simultané du code source (via Git), des jeux de données (*Data Versioning*) et des modèles. L'objectif est de pouvoir retracer exactement quels paramètres et quelles données ont généré une prédiction spécifique, une exigence non négociable en production [21, 22].
- **Le Registre de Modèles (*Model Registry*) :** Une fois entraîné et validé, le modèle est stocké dans un registre centralisé avec ses métadonnées et son lignage (*lineage*), prêt à être déployé [21, 23].
- **La Supervision en continu et les Boucles de rétroaction :** Le système intègre des outils d'observabilité pour surveiller la latence, la précision des prédictions et la dérive des données. Cette supervision alimente une boucle de rétroaction (*Feedback Loop*) qui déclenche automatiquement les pipelines de réentraînement (CT) dès que les performances chutent sous un seuil acceptable [21, 23].





1.1.2 Le Cloud Computing : Catalyseur de l'IA à grande échelle

1.1.2.1 Fondements et élasticité (Découplage Stockage/Calcul)

Le *Cloud Computing* (informatique en nuage) désigne la fourniture à la demande de ressources informatiques (puissance de calcul, stockage, bases de données, réseaux) via Internet, avec une tarification basée sur la consommation réelle (modèle *Pay-As-You-Go*). Dans le contexte de l'Intelligence Artificielle, l'industrie s'est massivement tournée vers le Cloud pour surmonter les limitations des infrastructures locales (*On-Premise*), qui s'avèrent souvent trop rigides et coûteuses face aux besoins exponentiels en calcul imposés par le traitement de données massives (Big Data) [10].

L'avantage architectural majeur du Cloud pour le MLOps est de permettre le découplage physique entre le stockage des données et la puissance de calcul [25]. Cette approche garantit une élasticité totale : les grappes de serveurs (clusters) peuvent être provisionnées dynamiquement lors de l'entraînement des modèles ou du prétraitement des données, puis détruites immédiatement après, optimisant ainsi l'allocation des ressources [6].

1.1.2.2 Modèles de services Cloud appliqués au MLOps

La force du Cloud Computing réside dans ses différents niveaux d'abstraction, permettant de structurer l'environnement technique selon les besoins spécifiques des équipes [10] :

- **IaaS (Infrastructure as a Service)** : Fournit les briques matérielles fondamentales virtualisées (serveurs, réseaux, disques). Pour le MLOps, l'IaaS permet aux ingénieurs de construire des architectures de calcul distribué sur-mesure, disposant de processeurs spécifiques (CPU, GPU, RAM) pour accélérer l'entraînement des réseaux de neurones complexes [10, 26].
- **PaaS (Platform as a Service)** : Offre des environnements de développement et de déploiement entièrement managés. C'est le modèle privilégié pour l'industrialisation de l'IA, car il élimine la gestion fastidieuse des serveurs sous-jacents. Il fournit des espaces de travail prêts à l'emploi (*workspaces*) et des services d'orchestration de conteneurs (ex : Kubernetes) ou de calcul distribué (ex : Apache Spark sur Amazon EMR) [10, 27].
- **SaaS (Software as a Service)** : Propose des applications logicielles complètes accessibles directement via une interface web ou une API. Dans le cadre de l'IA, cela inclut des modèles pré-entraînés prêts à être interrogés sans aucune gestion de l'infrastructure [10].

1.1.2.3 Défis sécuritaires et souveraineté des données

Malgré sa flexibilité, l'externalisation des données d'entreprise et des modèles d'IA sur des serveurs distants soulève des défis critiques en matière de sécurité, de confidentialité et de conformité réglementaire [6, 28] :





- **Souveraineté et Conformité (RGPD) :** Le Règlement Général sur la Protection des Données (RGPD) impose en Europe un cadre très strict sur la collecte et le stockage des données. Les architectes doivent s'assurer que les données sensibles sont hébergées dans des régions Cloud physiquement situées sur le territoire européen, garantissant ainsi la souveraineté numérique du système [29].
- **Isolation réseau et gestion des identités (IAM) :** L'architecture Cloud doit impérativement intégrer des mécanismes de chiffrement robustes (au repos et en transit). De plus, l'utilisation de Réseaux Privés Virtuels (VPC - *Virtual Private Cloud*) et la mise en place de politiques de gestion des identités et des accès (IAM - *Identity and Access Management*) appliquant le principe du "moindre privilège" sont indispensables pour sécuriser les environnements et empêcher toute fuite de données [30].

1.1.2.4 L'optimisation des coûts : L'approche FinOps

Le passage à l'échelle dans le Cloud introduit un risque d'explosion des coûts si les ressources ne sont pas gérées avec rigueur. Le FinOps (*Financial Operations*) est une pratique de gestion financière des ressources Cloud visant à maximiser la valeur métier. Dans le cadre du MLOps, cela se traduit par l'automatisation de la destruction des ressources inactives (clusters éphémères) et par l'exploitation d'instances de calcul à prix réduit (telles que les instances "Spot", qui exploitent la capacité excédentaire des fournisseurs Cloud) pour les charges de travail distribuées et tolérantes aux pannes [6, 25].

1.1.2.5 Fiabilité et supervision : Approche SRE (SLA, SLO, SLI)

L'industrialisation des modèles d'intelligence artificielle requiert l'adoption de pratiques d'ingénierie de la fiabilité rigoureuses (souvent inspirées du *Site Reliability Engineering* - SRE) pour éviter l'accumulation d'une « dette technique cachée » (*Hidden Technical Debt*) en production [24]. Pour garantir qu'un système MLOps répond aux exigences de fiabilité, l'industrie s'appuie sur trois concepts opérationnels fondamentaux :

- **SLA (Service Level Agreement - Accord de niveau de service) :** Il s'agit du contrat établi avec les consommateurs du modèle, définissant les attentes globales de performance (disponibilité, temps de réponse, etc.). Dans un système ML, l'établissement de SLA stricts est par exemple essentiel pour protéger le modèle contre des « consommateurs non déclarés » (*undeclared consumers*) qui pourraient le surcharger et compromettre la stabilité du système [24]. L'architecture Cloud dans son ensemble doit être alignée sur ces SLA [6].
- **SLO (Service Level Objective - Objectif de niveau de service) :** Il s'agit de l'objectif interne de performance. Les processus d'alimentation en données (en amont) et le système ML lui-même doivent être testés et supervisés en continu pour atteindre les objectifs de





niveau de service (SLO) fixés. Toute défaillance du modèle à respecter ces SLO doit être immédiatement propagée vers les systèmes dépendants en aval [24].

- **SLI (Service Level Indicator - Indicateur de niveau de service)** : C'est la métrique réelle et mesurable. Au-delà des SLI classiques de l'ingénierie logicielle (latence d'inférence, utilisation CPU/RAM), les systèmes d'IA introduisent des indicateurs très spécifiques : la dégradation de la précision algorithmique et le taux de dérive des données (*Data Drift*) [6, 22].

En production, le non-respect d'un SLO (mesuré par la chute critique d'un SLI, comme l'apparition d'une dérive de données) agit comme un déclencheur automatisé. Le pipeline MLOps doit être instrumenté pour détecter cette dégradation et ordonner l'exécution d'un cadre de réentraînement automatisé (*automated re-training framework*) avec de nouvelles données fraîchement collectées [6, 22]. À titre d'exemple pratique, une architecture MLOps robuste peut être configurée pour déclencher automatiquement un réentraînement dès lors qu'un indicateur de dérive statistique dépasse un seuil de tolérance prédéfini (ex : statistique KS > 0.20) [28].

1.1.2.6 Défis sécuritaires, conformité et protection des données

Malgré sa flexibilité, l'externalisation des données d'entreprise et des modèles d'IA sur des serveurs distants soulève des défis critiques en matière de sécurité, de confidentialité (protection des données personnelles et sensibles) et de conformité réglementaire [6]. Pour concevoir un système MLOps de qualité production, l'architecture doit impérativement intégrer des mécanismes de protection à plusieurs niveaux :

- **Souveraineté, résidence des données et conformité (ex : RGPD)** : Les cadres réglementaires (comme le RGPD en Europe) imposent des contrôles stricts sur la collecte, le traitement et le stockage des données. L'architecture doit intégrer dès sa conception la validation des permissions et de la confidentialité [6]. Pour s'y conformer, les ingénieurs doivent s'assurer que les données sensibles sont physiquement hébergées dans des régions Cloud spécifiques (ou via des solutions périphériques comme *AWS Local Zones*) afin de garantir la souveraineté numérique et de respecter les exigences strictes de résidence des données (*Data Residency*) inhérentes aux charges de travail réglementées [6].
- **Chiffrement et gestion des accès (IAM)** : Face aux législations extraterritoriales (telles que le *CLOUD Act* américain) et aux menaces de cybersécurité, l'infrastructure doit nativement intégrer des mécanismes de chiffrement robustes pour les données au repos et en transit (*Encryption at rest and in transit*) afin d'en assurer l'obfuscation [6]. De plus, la sécurisation des environnements exige la mise en place de politiques de Gestion des Identités et des Accès (IAM) appliquant rigoureusement le « principe du moindre privilège » (*Least privilege access*). Cela permet de cloisonner hermétiquement les réseaux, d'isoler les rôles d'exécution et de prévenir tout accès non autorisé aux environnements d'entraînement et d'inférence [6, 31].





1.2 État de l'art

Un véritable état de l'art implique d'analyser en profondeur la manière dont les leaders de l'industrie conçoivent et déploient leurs infrastructures pour passer à l'échelle.

Comme le soulignent Sculley et al. dans leurs travaux de recherche fondateurs, le code algorithmique ne représente qu'une infime fraction d'un système IA ; le reste est constitué d'une vaste infrastructure complexe de collecte, de vérification des données et de supervision [24]. L'absence d'une architecture rigoureuse conduit inévitablement à l'accumulation d'une « dette technique cachée », freinant l'innovation et exposant les entreprises à des risques critiques [24].

C'est pour surmonter cet obstacle que le paradigme du MLOps (*Machine Learning Operations*) s'est imposé, fusionnant l'ingénierie logicielle (DevOps), l'ingénierie des données (*Data Engineering*) et l'apprentissage automatique [21]. Toutefois, concevoir un système MLOps capable de traiter des volumes massifs de données exige de s'appuyer sur des modèles de conception (*design patterns*) durables [6].

Cette section propose donc une exploration structurée de ces standards technologiques :

- Nous disséquons les architectures de référence déployées sur le Cloud par cinq grandes organisations (Netflix, Capital One, Riot Games, Expedia Group, ainsi qu'un modèle standard basé sur l'Infrastructure as Code).
- Enfin, cette démarche analytique nous permettra de dresser un bilan objectif pour introduire et justifier l'architecture hybride retenue pour le compte de DESEMITTS LLC.

1.2.1 Cas d'étude 1 : Netflix – Architecture MLOps de référence, orchestration hybride et passage à l'échelle

1. Contexte et ampleur du défi technique

Netflix s'appuie sur le Cloud AWS pour diffuser des milliards d'heures de contenu à l'échelle mondiale et opérer sa plateforme analytique [32]. La création de modèles d'intelligence artificielle personnalisés exige d'entraîner un modèle spécifique pour chaque pays et chaque langue, ce qui génère une base de 2 000 à 4 000 modèles en production [25]. Lors des phases de recherche d'hyperparamètres (*hyperparameter sweeps*), ce volume explose pour atteindre simultanément entre 10^5 et 10^6 modèles [25]. Ces charges de travail massives, imprévisibles et extrêmement gourmandes en ressources (processeurs, mémoire, GPU) risquent à tout moment de saturer l'infrastructure et d'impacter les applications critiques en production [25].

Pour répondre à ce défi tout en préservant la productivité des *Data Scientists*, Netflix a conçu une architecture de référence modulaire, décomposée en plusieurs couches spécialisées et documentée dans ses retours d'expérience sur AWS [25].

2. Les couches de l'architecture de référence Netflix





A. Couche de gestion des données (Data Lake et Analytics)

La fondation de l'architecture repose sur un découplage strict entre la puissance de calcul et le stockage :

- **Data Lake unique** : L'intégralité des données (pétaoctets de vidéos et données tabulaires) est centralisée sur Amazon S3 [25].
- **Formats et Moteurs de requêtes** : Les données tabulaires sont stockées au format Parquet et exposées via des tables Iceberg [25]. Pour interroger ces données, Netflix utilise des moteurs de calcul distribué comme Apache Spark et Presto [25].
- **Fédération et Métadonnées** : L'accès est abstrait par *Genie*, un moteur de requêtes fédéré développé en interne (open source), tandis que la découverte des données et la gestion des métadonnées sont assurées par *Metacat* [25].
- **Bypass pour le ML** : Pour éviter les goulots d'étranglement des moteurs SQL lors de l'entraînement, des outils spécifiques permettent aux pipelines ML de se connecter directement à S3, saturant la bande passante réseau pour ingérer les données à très haute vitesse [25].

B. Couche d'orchestration des conteneurs et de calcul (Compute)

- **Conteneurisation via Titus / AWS Batch** : Netflix utilise son orchestrateur interne *Titus* (dont l'équivalent AWS est AWS Batch) pour lancer des conteneurs à grande échelle depuis les environnements locaux vers le Cloud [25].
- **Mécanisme de files d'attente (Isolation)** : Pour éviter que les pics d'entraînement n'impactent la production, les requêtes ML sont placées dans des files d'attente strictes. Ces files "fuient" (*leak*) progressivement les tâches vers Amazon ECS ou Kubernetes en fonction des priorités [25].
- **Optimisation FinOps par l'IA** : Les instances GPU étant onéreuses, Netflix a développé des modèles de Machine Learning secondaires chargés de prédire avec précision les ressources (CPU, RAM, GPU) requises par une tâche avant son lancement [25]. Cela permet un placement optimal des *jobs* sur les serveurs physiques, augmentant le taux d'utilisation et réduisant drastiquement les coûts opérationnels [25].

C. Couche d'orchestration des Workflows (Pipelines)

Les pipelines ML de Netflix peuvent durer des semaines (par exemple pour la vision par ordinateur) et nécessitent de transférer le calcul d'une instance à forte mémoire vers une instance GPU [25].

- **Netflix Conductor / AWS Step Functions** : L'orchestration asynchrone des différentes étapes du pipeline est gérée par *Netflix Conductor* (équivalent d'AWS Step Functions). Il gère la montée en charge requise pour exécuter des milliers de tests A/B en parallèle [25].





- **Signalisation des données (Data Signaling) :** L'architecture est purement événementielle. Au lieu d'interroger les bases de données à intervalles réguliers (*polling*), les pipelines ML sont déclenchés automatiquement par des événements (via Amazon EventBridge) dès qu'un pipeline ETL en amont a terminé l'écriture d'une table [25].

D. Couche d'environnement de développement et d'observabilité

- **Workspaces interactifs :** Les *Jupyter Notebooks* (Python, R) sont hébergés sur des instances Amazon SageMaker Notebooks [25]. Pour l'écosystème Scala/JVM, Netflix a développé et rendu open source le projet *Polynote* [25].
- **Supervision as Code :** La surveillance des modèles en production (temps d'exécution, comportement du flux) est réalisée via des notebooks transformés en tableaux de bord. Grâce aux outils open source *Papermill* et *Commuter*, les notebooks sont exécutés périodiquement et hébergés pour fournir des visualisations interactives via des bibliothèques comme Bokeh ou Plotly [25].

3. Le Framework Fédérateur : Metaflow

Pour lier toutes ces couches d'infrastructure sans forcer les *Data Scientists* à devenir des experts en ingénierie Cloud (Docker, Step Functions), Netflix a créé **Metaflow** [25]. Ce framework Python abstrait la complexité :

- L'utilisateur écrit du code Python idiomatique (TensorFlow, PyTorch, Scikit-learn) [25].
- Metaflow intègre une couche de données à haut débit depuis Amazon S3 [25].
- L'ajout d'un simple décorateur (ex : @batch) dans le code suffit pour que la fonction soit automatiquement exécutée sur une instance AWS Batch avec une mémoire ou des GPU spécifiés [25].
- En un clic, le code Python est compilé et exporté vers AWS Step Functions pour une orchestration en production [25].
- Metaflow versionne automatiquement toutes les métadonnées de chaque exécution, garantissant une reproductibilité totale [25].

4. Implémentation de référence sur AWS (Infrastructure as Code)

L'architecture de Netflix est reproductible via un modèle AWS CloudFormation (disponible via [netflix/metaflow-tools](https://github.com/netflix/metaflow-tools)) qui provisionne l'infrastructure complète [25].

- **Ressources provisionnées :** L'infrastructure déploie automatiquement un bucket S3 pour les artefacts, un environnement de calcul AWS Batch (avec mode de provisionnement EC2), un cluster Amazon ECS Fargate (avec accès au port 80), une base de données Postgres, DynamoDB, et un Network Load Balancer (NLB) [25].



- **Sécurité IAM stricte** : Des rôles AWS IAM appliquant le principe du moindre privilège sont générés pour garantir que, par exemple, AWS Batch ne puisse extraire de S3 que les données strictement nécessaires, de manière sécurisée [25].
- **Intégration et Catalogue** : L'ensemble est intégré à Amazon EventBridge pour la signalisation des données et à AWS Service Catalog pour mettre cette infrastructure à disposition des utilisateurs via un portail en libre-service [25].

Tableau Synthétique : Correspondance des composants (Netflix vs AWS)

TABLEAU 1.1 – Cartographie de l'Architecture de Référence MLOps (Netflix vs AWS)

Couche Architecturale	Outil Interne / Open Source Netflix	Équivalent / Service Managé AWS
Stockage (Data Lake)	S3, Iceberg, Parquet [25]	Amazon S3 [25]
Requêtage & Métadonnées	Genie, Metacat, Presto, Spark [25]	Amazon Athena, AWS Glue, EMR
Calcul & Conteneurisation	Titus [25]	AWS Batch, Amazon ECS (Fargate) [25]
Orchestration de Workflows	Netflix Conductor [25]	AWS Step Functions [25]
Signalisation (Événements)	Bus d'événements interne [25]	Amazon EventBridge [25]
Environnement (Workspace)	Polynote, Jupyter [25]	Amazon SageMaker Notebooks [25]
Supervision (Dashboards)	Papermill, Commuter [25]	Amazon CloudWatch
Framework d'abstraction	Metaflow [25]	Amazon SageMaker Pipelines

1.2.2 Cas d'étude 2 : Capital One – MLOps ultra-sécurisé, conformité et gouvernance dans le secteur bancaire (FSI)

1. Contexte et exigences du secteur financier

Le secteur des services financiers (Financial Services Industry - FSI) est soumis à des contraintes réglementaires extrêmes. Pour Capital One, une banque gérant plus de 100 millions de clients, l'adoption de l'Intelligence Artificielle générative ne pouvait se faire sans une infrastructure capable de prouver aux régulateurs que chaque décision est auditable, sécurisée et maîtrisée [33]. L'enjeu technique était de passer à l'échelle pour des dizaines de milliers d'employés (notamment plus de 10 000 agents du service client utilisant déjà l'IA générative dans leur flux de travail), tout en atténuant de manière stricte les risques de non-conformité [33].



2. Fondation architecturale : Migration "All-In" et Cloud-Native

Pour rendre possible une pratique MLOps performante et réactive, Capital One a pris une décision d'infrastructure radicale : fermer la totalité de ses centres de données physiques (passant d'environ 15 centres à zéro) pour migrer à 100 % vers le Cloud public AWS [33].

- **Centralisation sur Amazon S3 et "Data Gravity"** : Cette migration a permis de consolider l'intégralité des données propriétaires sur Amazon S3 [33]. Ce choix architectural a rapproché physiquement les données des grappes de calcul surpuissantes, créant un effet de « gravité des données » (*Data Gravity*) considéré par les architectes de la banque comme le précurseur indispensable au succès de l'IA [33].
- **Architecture Serverless** : Le développement MLOps de Capital One s'inscrit dans une démarche *Cloud-native* et *Serverless*, éliminant la gestion des serveurs sous-jacents pour les scientifiques des données et offrant une élasticité instantanée [33].

3. La Gouvernance MLOps : Le modèle des « Trois Lignes de Défense »

Contrairement aux entreprises technologiques classiques, l'architecture MLOps d'une banque doit intégrer la conformité de manière systémique. Capital One a institutionnalisé un modèle de gouvernance strict à "trois lignes de défense" [33] :

- **1^{ère} Ligne (Développeurs et MLOps)** : Les ingénieurs et chercheurs conçoivent les solutions, en intégrant des pratiques de responsabilité algorithmique dès la conception (*Privacy by design*) [33].
- **2^{ème} Ligne (Bureau des Risques / *Model Risk Office*)** : Cette équipe indépendante de la première ligne évalue, conteste et valide chaque modèle avant son déploiement. Des tests de référence internes et externes rigoureux sont exécutés pour s'assurer que le modèle respecte les normes de l'entreprise et la loi [33].
- **3^{ème} Ligne (Audit interne et externe)** : Une fonction d'audit inspecte le système en continu et fait un rapport direct au conseil d'administration. Les régulateurs gouvernementaux interviennent au minimum chaque trimestre pour vérifier que les contrôles entre la première et la deuxième ligne sont effectifs [33].

4. Couche de Sécurité Active : L'intégration des "Guardrails"

Pour déployer des modèles génératifs sans risquer d'exposer des données ou de subir des attaques, l'architecture MLOps intègre nativement des garde-fous, notamment *NeMo Guardrails* (développé par NVIDIA), au cœur de son pipeline d'inférence [33].

- **Sécurisation en Entrée** : Les garde-fous interceptent et bloquent les requêtes malveillantes (tentatives de *Jailbreak* ou d'injection de requêtes) de manière dynamique, en exploitant les bases de données de menaces mondiales mises à jour en continu [33].
- **Sécurisation en Sortie et Human-in-the-Loop** : Les réponses sont filtrées en temps réel pour garantir que le modèle reste sur le sujet autorisé et que le contenu généré est sûr.





Pour les cas d'usage à haut risque impliquant des clients finaux, un agent humain valide les sorties avant toute transmission (*Human-in-the-Loop*) [33].

5. Qualité des données, Personnalisation (Fine-Tuning) et FinOps

Les architectes de Capital One ont constaté que les modèles généralistes prêts à l'emploi n'offraient pas la précision requise pour les spécificités de la culture d'entreprise bancaire [33].

- **Qualité vs Volume** : L'architecture insiste sur la qualité des données (complétude, nettoyage, cohérence) plutôt que sur le simple volume. Des pipelines automatisés assurent la qualité des données d'entraînement, étape cruciale pour éviter que les modèles ne se basent sur des variations aberrantes [33].
- **PEFT et LoRA** : La plateforme MLOps intègre des pipelines de personnalisation avancés : pré-entraînement continu, ajustement fin supervisé (SFT), et des méthodes paramétriques efficaces comme LoRA (*Low-Rank Adaptation*) et QLoRA [33].
- **Optimisation FinOps massive de l'inférence** : L'inférence représentant le coût continu le plus élevé, l'infrastructure est rigoureusement optimisée. Cette ingénierie de pointe a permis à Capital One de réduire les coûts d'inférence de trois ordres de grandeur (un facteur 1 000) en l'espace de seulement 20 mois [33].

6. Supervision Post-Déploiement et Effet "Heisenberg"

Chez Capital One, le déploiement d'un modèle n'est considéré que comme « la première marche de l'escalier vers le paradis de l'IA » [33].

- **L'effet Heisenberg et la dérive induite** : L'entreprise observe que le simple fait de déployer un modèle modifie le comportement des utilisateurs, ce qui altère rapidement la distribution des données réelles par rapport à la phase d'entraînement. Le pipeline MLOps gère donc une observabilité continue pour capturer cette nouvelle réalité et déclencher des réentraînements [33].
- **Validation en ligne (A/B Testing)** : Parce que les tests hors ligne sur des données anciennes ne sont souvent pas de bons prédicteurs des performances réelles, la plateforme MLOps est instrumentée pour exécuter des tests en ligne limités (A/B testing) afin de valider continuellement la performance des algorithmes en production [33].

Tableau Synthétique : Correspondance des composants (Capital One)





TABLEAU 1.2 – Cartographie de l'Architecture MLOps Sécurisée (Secteur Bancaire / FSI)

Exigence / Couche Architecturale	Solutions et Pratiques implémentées
Stockage (Data Gravity)	Migration totale sur AWS, stockage massif et centralisé sur Amazon S3 [33]
Sécurisation (Entrée/Sortie)	Déploiement de <i>NVIDIA NeMo Guardrails</i> pour filtrer le <i>Jail-break</i> et garantir la sécurité du contenu (avec <i>Human-in-the-Loop</i> si nécessaire) [33]
Gouvernance (Conformité)	Validation stricte à 3 lignes de défense : 1. Développeurs, 2. Model Risk Office, 3. Auditeurs et Régulateurs [33]
Ajustement des modèles	Personnalisation via <i>Fine-Tuning</i> , LoRA et QLoRA [33]
Supervision	Observabilité continue pour contrer la dérive des données (Effet Heisenberg) et Tests A/B en production [33]

1.2.3 Cas d'étude 3 : Riot Games – Inférence et déploiement mondialisé à très faible latence (Edge Computing)

1. Contexte et exigence de latence critique

Pour des jeux compétitifs d'envergure mondiale tels que *League of Legends*, l'expérience du joueur et la réactivité de l'infrastructure sont des piliers fondamentaux. Face à l'augmentation de la base de joueurs, Riot Games a identifié l'opportunité de moderniser son infrastructure tout en maintenant une empreinte mondiale [26]. Le défi d'ingénierie consistait à déployer des serveurs de jeu et des charges de travail applicatives capables de s'adapter dynamiquement à l'activité des joueurs, tout en éliminant les délais de transmission inhérents aux architectures Cloud trop centralisées [26].

2. La stratégie de déploiement hybride et standardisée

Pour contourner les limites physiques de la latence, Riot Games a repensé la topologie de son infrastructure en adoptant un modèle de déploiement mondial standardisé à trois niveaux d'abstraction [26] :

- **Niveau 1 (Régions AWS) :** L'infrastructure privilégie en premier lieu les *Régions AWS* classiques lorsqu'elles sont disponibles et suffisamment proches des bassins de joueurs [26].
- **Niveau 2 (AWS Local Zones) :** Lorsque les Régions sont trop éloignées, Riot Games s'appuie sur les *AWS Local Zones*. Celles-ci rapprochent les services fondamentaux d'AWS (calcul, stockage) directement dans les grandes zones métropolitaines du monde entier, offrant la même élasticité à la demande mais avec une latence considérablement réduite [26].





- **Niveau 3 (AWS Outposts) :** Dans les zones géographiques où ni les Régions ni les Local Zones ne sont (encore) implantées, l'entreprise déploie *AWS Outposts* (des baies de serveurs physiques AWS installées dans des centres de données locaux) [26].

Au fur et à mesure que de nouvelles *Local Zones* sont inaugurées dans le monde, Riot Games y migre progressivement ses charges de travail depuis *Outposts*, évoluant vers un modèle où AWS gère entièrement à la fois l'infrastructure et les installations physiques [26].

3. Conteneurisation et standardisation des calculs

L'agilité de ce déploiement massif repose sur la conteneurisation. Riot Games utilise de manière intensive **Amazon EKS (Elastic Kubernetes Service)**, un service entièrement managé qui permet d'automatiser et d'exécuter des clusters Kubernetes de manière transparente dans le Cloud [26]. Ces clusters tournent sur des instances **Amazon EC2**, offrant à Riot Games une plateforme de calcul disposant du choix le plus large en matière de processeurs, de stockage et de mise en réseau pour adapter la puissance à chaque charge de travail [26].

4. Réseau dédié et contournement d'Internet

Déployer des serveurs en périphérie (*Edge*) ne suffit pas si les données doivent transiter par les routes imprévisibles de l'Internet public. Pour connecter ces déploiements distribués de manière sécurisée et ultra-rapide à son propre réseau principal (le backbone *Riot Direct*), Riot Games utilise **AWS Direct Connect** [26]. Ce service fournit une connexion réseau physique et dédiée directement reliée à l'infrastructure d'AWS, garantissant une bande passante constante et une latence minimale [26].

5. Abstraction de l'infrastructure et collaboration DevOps

L'un des bénéfices majeurs de cette architecture de référence est la transparence opérationnelle qu'elle offre aux développeurs. Comme l'indique David Press, responsable technique pour *League of Legends* : « *En tant qu'équipe de jeu, nous n'avons pas à nous demander si notre jeu tourne dans une AWS Local Zone, un AWS Outpost ou une Région AWS — c'est exactement la même chose pour nous* » [26].

Cette standardisation fournit une interface, une facturation et des outils totalement uniformisés. Elle libère les développeurs de la gestion de l'infrastructure, leur permettant d'investir leur temps dans l'amélioration de l'expérience joueur [26]. De plus, cette approche est soutenue par une collaboration étroite (planification des capacités, coordination des lancements) avec les architectes de solutions et les gestionnaires de comptes techniques d'AWS, agissant comme de véritables collaborateurs intégrés à l'équipe [26].

Tableau Synthétique : Composants de l'Architecture Mondiale (Riot Games)





TABLEAU 1.3 – Cartographie de l'Architecture Cloud à très faible latence (Riot Games / AWS)

Exigence / Couche Architecturale	Solutions et Services AWS implémentés
Déploiement en périphérie (Edge)	Stratégie combinée priorisant les Régions AWS, puis les <i>AWS Local Zones</i> (pour les métropoles mondiales), et enfin <i>AWS Outposts</i> [26]
Orchestration & Conteneurisation	Utilisation centralisée d' <i>Amazon EKS</i> (Kubernetes managé) tournant sur des instances de calcul <i>Amazon EC2</i> [26]
Interconnexion & Réseau Backbone	Connexion dédiée du réseau propriétaire (<i>Riot Direct</i>) à l'infrastructure via <i>AWS Direct Connect</i> [26]
Abstraction & Agilité	Interface et outils unifiés quel que soit le lieu d'exécution physique (Region, Local Zone ou Outpost), déchargeant les développeurs de la gestion matérielle [26]

1.2.4 Cas d'étude 4 : Expedia Group – Standardisation MLOps, conteneurisation et inférence multi-région

1. Contexte, fragmentation et défi de la latence globale

Le géant du voyage Expedia Group opère plus de 20 marques mondiales (Expedia, Hotels.com, Vrbo, etc.) et s'appuie sur plus de 650 équipes d'ingénierie pour soutenir 200 sites de voyage à travers le monde [29]. L'entreprise a été confrontée à un défi critique sur son service de suggestion (ESS – *Expedia Suggest Service*), un algorithme basé sur la localisation et l'historique d'achat des clients qui affiche des prédictions dès que l'utilisateur commence à taper sa recherche [34].

Cependant, l'infrastructure historique du groupe était centralisée dans un centre de données unique situé à Chandler, en Arizona. Cet éloignement physique générait une latence réseau inacceptable de 700 millisecondes pour les clients de la région Asie-Pacifique, entraînant des erreurs de page et l'abandon des utilisateurs [34]. L'enjeu était de moderniser l'infrastructure pour standardiser les déploiements de ces centaines d'équipes et rapprocher les modèles d'IA au plus près des utilisateurs finaux [29, 34].

2. Standardisation des déploiements via AWS Service Catalog

Pour unifier les pratiques d'ingénierie sans freiner l'innovation de ses 650 équipes, Expedia a mis en place une plateforme standardisée :

- **Plateforme DBaaS et Gouvernance** : L'entreprise a développé *Cerebro*, une plateforme de base de données en tant que service (DBaaS). Grâce à **AWS Service Catalog**, une petite équipe d'ingénieurs centraux a pu déployer ce catalogue à l'échelle de l'organisation, permettant aux équipes de provisionner rapidement des environnements tout en respectant nativement les meilleures pratiques et la gouvernance du groupe [29].





- **Modèle de déploiement standardisé** : En créant des modèles d'infrastructure reproductibles, les équipes de science des données ne sont plus limitées par des contraintes de débit ou de capacité CPU, et peuvent provisionner l'infrastructure de nouvelles initiatives de manière autonome [34].

3. Architecture de données massives et Conteneurisation

Pour alimenter ses modèles d'intelligence artificielle, l'architecture a été migrée vers un modèle orienté conteneurs et services managés :

- **Ingestion et orchestration Big Data** : Expedia a construit un système flexible pour ingérer de hauts volumes de données (en temps réel et par lots) en orchestrant **AWS Glue**, **Amazon DynamoDB**, **Amazon Aurora** et **Amazon CloudWatch** [29].
- **Conteneurisation via Amazon ECS** : La migration des applications vers des plateformes conteneurisées basées sur **Amazon Elastic Container Service (ECS)** a permis au groupe de se libérer de la gestion des serveurs sur site, de ne payer que pour les ressources réellement utilisées, et d'ajouter de la capacité dynamiquement [29, 34].
- **Calcul intensif (Deep Learning)** : Pour l'entraînement de ses algorithmes de traitement d'images, le groupe utilise des instances **Amazon EC2 P3** (optimisées GPU). Ces modèles analysent des dizaines de millions de photos avec une précision supérieure à 99 %, afin de sélectionner automatiquement les images d'hôtels les plus pertinentes [29].

4. Réplication Multi-Région et réduction drastique de la latence

Pour résoudre le défi de la latence de son service de suggestion (ESS), Expedia a repensé son modèle de déploiement mondial :

- **Déploiement mondial et Haute Disponibilité** : Le service a d'abord été déployé en Asie-Pacifique, puis rapidement répliqué dans les régions États-Unis et Europe. L'architecture s'appuie sur plusieurs *Zones de Disponibilité (AZ)* pour garantir la reprise après sinistre et une disponibilité continue [34]. Les données elles-mêmes sont sécurisées via **Amazon S3** et ses mécanismes de réplication automatique [29].
- **Gains de performance (Inférence)** : Le rapprochement des points d'inférence au plus près des utilisateurs a permis de faire chuter la latence moyenne du réseau de 700 millisecondes à **moins de 50 millisecondes** [34]. À l'échelle globale, cette plateforme traite plus de **600 milliards de prédictions IA par an** à partir de 70 pétaoctets de données, avec des temps de réponse sous la milliseconde [29].

5. Sécurisation et gouvernance des données personnelles (PII)

L'industrie du voyage traitant quotidiennement des données hautement sensibles, la sécurité du Data Lake est une priorité absolue :

- **Protection via Amazon Macie** : Face à l'augmentation des exigences réglementaires autour des données personnelles (PII) et de santé (PHI), Expedia applique une approche de



sécurité multicouche. Le groupe exploite **Amazon Macie**, un service qui utilise lui-même le Machine Learning pour découvrir, surveiller et protéger automatiquement les données sensibles stockées dans les environnements AWS [29].

Tableau Synthétique : Composants MLOps et Data (Expedia Group)

TABLEAU 1.4 – Cartographie de l'Architecture et des Services Cloud (Expedia Group)

Exigence / Couche Architecturale	Solutions et Services AWS implémentés
Standardisation & Libre-service	Mise à disposition d'environnements (ex : DBaaS <i>Cerebro</i>) validés et standardisés via <i>AWS Service Catalog</i> [29]
Ingestion de données	Pipelines Big Data temps réel et batch via <i>AWS Glue</i> , <i>DynamoDB</i> et <i>Amazon Aurora</i> [29]
Orchestration & Conteneurisation	Migration vers des applications conteneurisées et pilotées par <i>Amazon ECS</i> [29, 34]
Performance & Faible Latence	Réplication mondiale sur de multiples <i>Availability Zones</i> faisant chuter la latence de 700 ms à < 50 ms [34]
Gouvernance & Sécurité des PII	Détection et supervision en continu des données sensibles (PII/PHI) par <i>Amazon Macie</i> [29]

1.2.5 Cas d'étude 5 : Plateforme MLOps sécurisée basée sur Terraform, GitHub et Amazon SageMaker

1. Contexte et exigence d'une architecture multi-comptes

Pour industrialiser des cas d'usage d'apprentissage automatique de manière sécurisée et reproductible, les entreprises doivent s'appuyer sur des plateformes MLOps garantissant la séparation des environnements (Développement, Préproduction, Production) et l'observabilité de bout en bout [30]. Bien qu'Amazon SageMaker propose des outils CI/CD natifs, de nombreuses organisations exigent d'utiliser leurs propres standards de gestion de code et d'infrastructure pour éviter la fragmentation technologique [35]. Ce cas d'étude publié par AWS détaille une architecture de référence déployant une plateforme MLOps sécurisée reposant intégralement sur l'Infrastructure as Code (Terraform) et l'automatisation via GitHub Actions [30].

2. Fondation : Infrastructure as Code (IaC) via Terraform

L'ensemble de l'infrastructure Cloud sous-jacente est codée, versionnée et provisionnée à l'aide de Terraform, garantissant que les environnements de développement et de production sont strictement identiques [30]. L'architecture s'appuie sur sept modules Terraform réutilisables et hautement sécurisés [30] :





- **Networking (Réseau et VPC) :** Création d'un *Virtual Private Cloud* (VPC) isolé, comprenant des sous-réseaux privés, des groupes de sécurité, des passerelles NAT et de multiples *VPC Endpoints*. Cela permet à SageMaker de communiquer avec les autres services AWS de manière sécurisée sans transiter par l'Internet public [30].
- **Sécurité (IAM et KMS) :** Déploiement de clés de chiffrement gérées par le client via *AWS Key Management Service* (KMS) pour chiffrer les données. Des rôles *AWS IAM* basés sur le principe du moindre privilège sont générés spécifiquement pour les utilisateurs et les services SageMaker [30].
- **Gestion de l'état (S3) :** Conformément aux bonnes pratiques de l'IaC, l'état de l'infrastructure Terraform est verrouillé et stocké de manière centralisée sur Amazon S3 [30].

3. Intégration CI/CD : GitHub, GitHub Actions et AWS CodeStar

Le code source des modèles et la configuration de l'infrastructure sont hébergés sur **GitHub**. L'automatisation du cycle de vie est assurée par **GitHub Actions**, qui orchestre la construction, les tests et le déploiement de manière agnostique [30, 35].

- **AWS CodeStar Connection :** Pour permettre à AWS de communiquer de manière sécurisée avec le dépôt GitHub, une connexion *AWS CodeStar* est établie. Le jeton d'accès personnel (PAT) de GitHub est stocké de manière chiffrée dans *AWS Secrets Manager* [35].
- **Validation manuelle (GitHub Environments) :** Pour le déploiement des modèles vers les comptes de Préproduction et de Production, l'architecture utilise la fonctionnalité *Environments* de GitHub, qui impose l'approbation manuelle d'un réviseur désigné avant d'autoriser la mise en production [35].

4. Modèles de déploiement en libre-service (SageMaker Projects & Service Catalog)

Afin d'offrir une expérience en libre-service aux *Data Scientists* sans compromettre la sécurité, l'architecture utilise **Amazon SageMaker Projects** couplé à **AWS Service Catalog** [30, 35].

- **Portefeuille de modèles personnalisés :** Les ingénieurs MLOps créent un portefeuille de modèles CloudFormation personnalisés dans Service Catalog (ex : modèle d'entraînement pour LLM, modèle de déploiement BYOC - *Bring Your Own Container*) [30].
- **Clonage automatisé via Lambda :** Lorsqu'un utilisateur initie un nouveau projet dans SageMaker Studio, une fonction *AWS Lambda* est automatiquement invoquée pour cloner un dépôt GitHub privé contenant le code de base du modèle sélectionné, préparant ainsi l'espace de travail instantanément [30].

5. Orchestration du Workflow et Registre de modèles (SageMaker Pipelines)

Une fois le code poussé sur GitHub, GitHub Actions déclenche l'exécution d'**Amazon SageMaker Pipelines**. Ce service orchestre les flux de travail d'apprentissage automatique de manière native sur AWS [30, 35] :





- **Exécution du pipeline (DAG) :** Le graphe d'exécution gère la préparation des données, l'entraînement du modèle et son évaluation de manière automatisée [35].
- **SageMaker Model Registry :** Si le modèle dépasse le seuil de précision requis, il est automatiquement enregistré dans le registre de modèles centralisé (*Model Registry*) d'AWS avec son lignage de données complet et ses métriques [30, 35].
- **Déclenchement événementiel (EventBridge) :** *Amazon EventBridge* surveille le registre. Lorsque le statut d'un modèle passe à "Approuvé", EventBridge déclenche une fonction Lambda qui lance à son tour le workflow de déploiement dans GitHub Actions pour créer les *Endpoints* de staging ou de production [35].

Tableau Synthétique : Composants MLOps (Terraform, GitHub, SageMaker)

TABLEAU 1.5 – Cartographie de l'Architecture de Référence (AWS, Terraform, GitHub)

Couche / Exigence Technique	Solutions et Services implémentés
Infrastructure as Code (IaC)	Provisionnement du réseau (VPC, Endpoints), de la sécurité (IAM, KMS) et du stockage codé via <i>Terraform</i> [30]
Intégration & Déploiement (CI/CD)	Orchestration complète via <i>GitHub Actions</i> avec approbation manuelle (<i>Environments</i>) [30, 35]
Gouvernance & Libre-service	Mise à disposition des templates projets via <i>AWS Service Catalog</i> connectés à GitHub via <i>AWS Lambda</i> et <i>CodeStar</i> [30, 35]
Orchestration des Workflows	Apprentissage automatisé via <i>Amazon SageMaker Pipelines</i> (graphe DAG) [30, 35]
Registre & Automatisation	Versionnage dans <i>SageMaker Model Registry</i> . Le passage en production est détecté par <i>Amazon EventBridge</i> [35]

1.2.6 Bilan de l'état de l'art et Positionnement technologique du projet

1. Bilan de l'état de l'art

L'analyse des architectures de référence déployées par les leaders de l'industrie (Netflix, Capital One, Riot Games, Expedia) met en évidence un consensus clair : pour qu'un modèle soit robuste et passe à l'échelle, il est impératif de découpler la gestion massive des données (Data Lake, calcul distribué) de l'orchestration du cycle de vie du Machine Learning (CI/CD, déploiement) [6, 25].

2. Positionnement de notre projet

La problématique du client concerne l'industrialisation d'un modèle de vision par ordinateur face à une volumétrie massive et croissante de données non structurées (images agricoles).



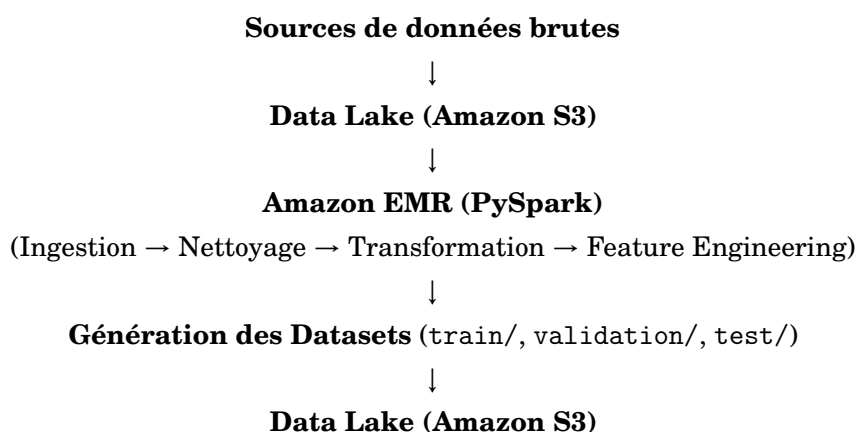


Pour y répondre, **notre positionnement stratégique s'appuie exactement sur l'architecture de référence du Cas d'étude n°5 (AWS, Terraform, GitHub et SageMaker) [30, 35]**. Nous y apportons une simplification (déploiement sur un environnement mono-compte pour réduire la complexité administrative) et nous scindons l'architecture en deux parties distinctes et indépendantes pour absorber la charge de données.

Partie 1 : Ingénierie des données (Data Engineering)

L'architecture de traitement des données est totalement isolée et dédiée à la préparation. Le cluster **Amazon EMR** (avec PySpark) ne fait **aucun Machine Learning**. Il a pour unique responsabilité de traiter la volumétrie massive en amont et de produire des jeux de données propres.

Le flux de traitement s'opère selon la séquence suivante :



À l'issue de cette première partie, les données agricoles sont nettoyées, transformées et stockées de manière structurée dans le Data Lake S3, prêtes à être consommées par l'équipe Data Science.

Partie 2 : Plateforme MLOps (Standard AWS)

Une fois les données propres disponibles sur S3, nous utilisons **exactement l'architecture MLOps de référence proposée par AWS [30, 35]**.

Le déploiement de l'infrastructure et des pipelines repose sur la chaîne de CI/CD suivante :

GitHub → GitHub Actions → Terraform → AWS → Amazon SageMaker Studio

Le flux de travail du Data Scientist :

1. **Initialisation** : Le Data Scientist ouvre son environnement de développement dans *Amazon SageMaker Studio*.
2. **Accès aux données** : Il retrouve automatiquement les données prêtes à l'emploi (les répertoires train/, validation/ et test/ générés par EMR) directement depuis le bucket S3.

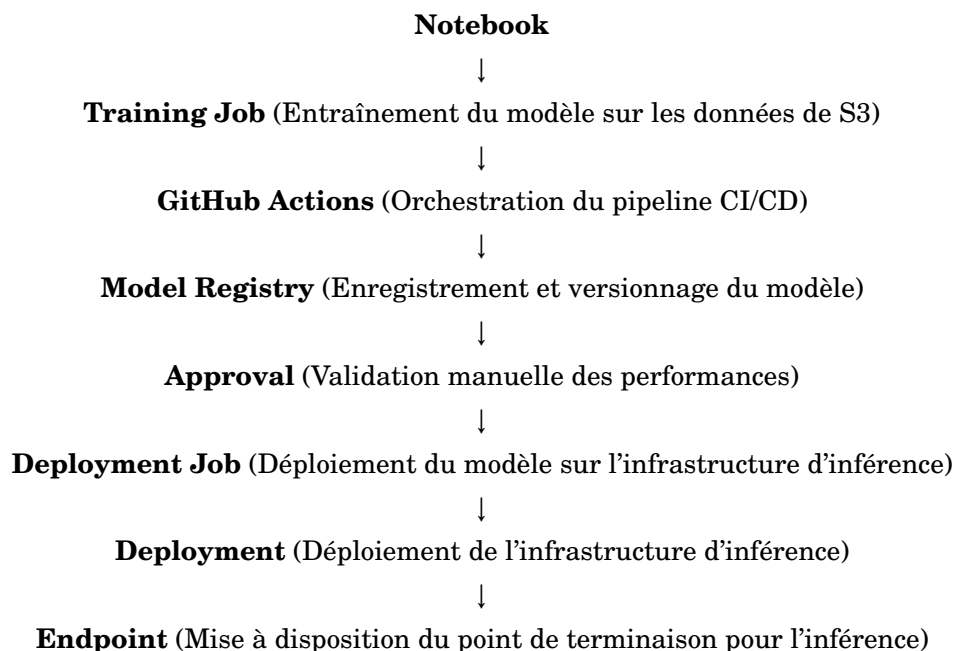




3. **Développement** : Il développe et expérimente son modèle depuis son Notebook Jupyter.

Le pipeline de mise en production :

Une fois le Notebook terminé et le code poussé, le pipeline MLOps natif (piloté par GitHub Actions et SageMaker Pipelines) prend le relais avec le flux d'exécution standard suivant [30, 35] :



Cette séparation stricte garantit que la plateforme MLOps reste agile et performante (Partie 2), tandis que la lourdeur du traitement de la volumétrie croissante des images est intégralement absorbée par la puissance brute du cluster Big Data (Partie 1). Le chapitre suivant sera consacré à la méthodologie de ce projet, en détaillant l'analyse des besoins, les diagrammes modélisant le système, et la conception logique et physique de cette architecture Cloud.



MÉTHODOLOGIE DE CONCEPTION

Après avoir dressé l'état de l'art et défini notre positionnement technologique (une architecture hybride séparant l'ingénierie des données massives sur Amazon EMR et la plateforme MLOps sur Amazon SageMaker), ce chapitre détaille la démarche adoptée pour concevoir cette solution. Nous y aborderons la planification du projet, l'analyse approfondie des besoins et des acteurs, pour aboutir à la modélisation logique et physique de l'architecture déployée.

Sommaire

2.1	Planification du projet	26
2.2	Analyse du problème et définition des besoins	27
2.2.1	Analyse du problème	27
2.2.2	Besoins fonctionnels	27
2.2.3	Besoins non fonctionnels	29
2.2.4	Acteurs du système	31
2.2.5	Diagramme de cas d'utilisation	32
2.3	Conception de la solution	33
2.3.1	Architecture logique	33
2.3.2	Architecture physique	34
2.4	Résumé du chapitre	36



2.1 Planification du projet

La mise en place d'une infrastructure MLOps Cloud robuste nécessite une démarche itérative et incrémentale. Contrairement au développement logiciel classique, le passage à l'échelle d'un système d'intelligence artificielle implique une gestion complexe et évolutive des données (*Data Engineering*), de l'infrastructure en tant que code (IaC) et du cycle de vie des modèles.

Pour mener à bien ce projet, nous avons adopté une méthodologie Agile (inspirée du cadre Scrum), particulièrement adaptée aux environnements Cloud où l'expérimentation, la flexibilité et l'adaptation continue sont primordiales. Le projet s'est étalé sur une durée globale d'environ sept mois, découpée en **12 sprints de 2,5 semaines** chacun.

Cette planification s'est articulée autour de grandes phases logiques, reflétant directement notre double positionnement architectural :

- **Sprints 1 à 3 (Cadrage et Modélisation)** : Analyse des besoins fonctionnels et non fonctionnels, modélisation de l'architecture hybride et définition des exigences de sécurité et de conformité Cloud.
- **Sprint 4 (Ingénierie des données - Partie 1)** : Configuration du Data Lake (Amazon S3) et provisionnement du cluster Big Data éphémère (Amazon EMR). Développement des scripts PySpark pour l'ingestion, le nettoyage et le *Feature Engineering*.
- **Sprints 5 à 6 (Tests et validation intermédiaires)** : Évaluation des capacités de calcul distribué. Afin de tester et valider formellement nos scripts PySpark sans attendre les données privées finales, nous avons utilisé le jeu de données public *fruits-360* (disponible sur Kaggle). Cela a permis de prouver la scalabilité du prétraitement des images sur Amazon EMR avant de passer à l'étape suivante.
- **Sprints 7 à 9 (Plateforme MLOps - Partie 2)** : Mise en place de l'Infrastructure as Code avec Terraform, configuration des pipelines CI/CD via GitHub Actions et déploiement de l'environnement de développement Amazon SageMaker Studio pour l'équipe Data Science.
- **Sprints 10 à 12 (Validation architecturale et Intégration)** : Démonstration de la continuité de la chaîne de bout en bout. Nous avons validé le déclenchement des pipelines iac pour l'infrastructure SageMaker, VPC, User Roles, etc, via GitHub Actions, et finalisé la documentation technique du système.

L'adoption de ces sprints nous a permis de valider chaque brique technologique indépendamment (d'abord la préparation des données sur un dataset de référence, puis l'orchestration du modèle) avant de les assembler, réduisant ainsi les risques d'intégration.





2.2 Analyse du problème et définition des besoins

2.2.1 Analyse du problème

Le modèle de vision par ordinateur développé par le client de DESEMITs a prouvé son efficacité en local sur un petit jeu de donnée pour la détection d'anomalies sur les cultures agricoles. Cependant, son exécution reposait jusqu'ici sur des scripts locaux monolithiques. Face à la croissance exponentielle du volume d'images collectées (haute résolution), cette approche atteint ses limites physiques : saturation de la mémoire vive (RAM), temps de calcul prohibitifs et impossibilité de versionner ou de suivre la dégradation du modèle au fil des saisons agricoles.

Pour rendre ce modèle exploitable de manière durable, il est impératif de concevoir une infrastructure Cloud capable d'ingérer ces données massives, de distribuer la charge de prétraitement, et de fournir un cadre standardisé pour le réentraînement et le déploiement du modèle en production.

2.2.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les actions concrètes et spécifiques que notre architecture hybride doit être capable d'exécuter. Pour répondre au défi de la volumétrie et de l'automatisation, ces besoins sont structurés autour de quatre piliers fondamentaux.

2.2.2.1 Pilier 1 : Stockage centralisé et Gestion du Data Lake

Le système doit fournir un référentiel unique pour rompre avec la fragmentation des données locales. Ce stockage doit permettre :

- L'ingestion et l'archivage sécurisé de volumes massifs d'images agricoles brutes.
- Le stockage organisé des jeux de données générés après prétraitement (répertoires train/, validation/, test/).
- La centralisation des artefacts de modèles et des journaux d'exécution.

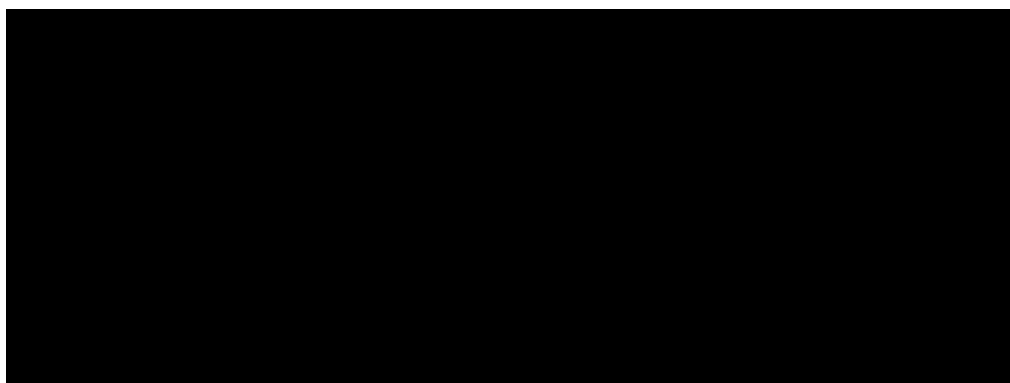


FIGURE 2.1 – Modélisation fonctionnelle du flux de données vers le Data Lake





2.2.2.2 Pilier 2 : Prétraitement distribué (Ingénierie des données)

Face à l’incapacité d’une machine unique à traiter les images en un temps raisonnable, le système doit impérativement intégrer un moteur de calcul distribué pour l’équipe Data.

TABLEAU 2.1 – Exigences fonctionnelles liées au prétraitement des images agricoles

Fonctionnalité	Description de l’action système
Nettoyage	Filtrage automatique des images corrompues ou inexploitable avant analyse.
Transformation	Redimensionnement et normalisation des matrices d’images en parallèle.
Feature Engineering	Extraction de caractéristiques visuelles (via Analyse en Composantes Principales - ACP) pour réduire la dimensionnalité des données.
Génération de Data-sets	Sauvegarde automatisée des données propres et formatées vers le Data Lake.

2.2.2.3 Pilier 3 : Environnement de développement et CI/CD

L’architecture doit faciliter le travail quotidien des équipes techniques (Data Scientists et ingénieurs MLOps) tout en garantissant la reproductibilité du code.

- **Espace de travail en libre-service** : Fournir un environnement de développement interactif (Notebooks), directement connecté au Data Lake, sans exiger de configuration d’infrastructure de la part du Data Scientist.
- **Automatisation (IaC et CI/CD)** : Le déploiement de l’infrastructure sous-jacente et l’exécution des tests doivent être automatisés via des pipelines d’intégration et de livraison continues déclenchés à chaque modification du code source.



FIGURE 2.2 – Flux fonctionnel d’Intégration et de Déploiement Continus (CI/CD)





2.2.2.4 Pilier 4 : Orchestration ML, Registre et Déploiement

La plateforme doit encadrer le cycle de vie du modèle d'intelligence artificielle de manière reproductible et traçable, en isolant cette étape du traitement de la donnée brute.

TABLEAU 2.2 – Exigences fonctionnelles liées à la Plateforme MLOps

Fonctionnalité	Description de l'action système
Orchestration (Pipelines)	Exécution séquentielle et automatisée de l'entraînement du modèle sur les données propres, suivie de son évaluation.
Registre de Modèles	Stockage centralisé et versionnage de chaque modèle entraîné, accompagné de ses métriques de performance et de son lignage.
Approbation	Mécanisme conditionnant le passage d'un modèle candidat vers la production (statut <i>Approved</i> ou <i>Rejected</i>).
Déploiement (Inférence)	Mise à disposition du modèle validé sous forme de point de terminaison (<i>Endpoint</i>) pour réaliser des prédictions sur de nouvelles images.

2.2.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité, de performance et de sécurité auxquels notre architecture Cloud doit impérativement satisfaire. Pour assurer la pérennité du système de vision par ordinateur du client, ces exigences ont été catégorisées en quatre axes majeurs :

2.2.3.1 Axe 1 : Scalabilité et Élasticité (Passage à l'échelle)

Face à la volumétrie massive et imprévisible des images agricoles, l'infrastructure doit être capable de s'adapter dynamiquement à la charge de travail.

- **Traitement Big Data** : Le cluster de calcul (*Amazon EMR*) doit pouvoir dimensionner automatiquement ses nœuds de travail (*Worker Nodes*) pour absorber le flux d'images lors d'une campagne de récolte, sans dégradation du temps de traitement.
- **Inférence ML** : Les points de terminaison (*Endpoints*) d'*Amazon SageMaker* doivent intégrer des politiques d'élasticité (*Auto Scaling*) pour gérer les pics de requêtes de prédiction en production, tout en maintenant une faible latence.

2.2.3.2 Axe 2 : Optimisation financière (Approche FinOps)

Étant donné l'intensité des calculs requis pour le traitement d'images haute résolution et l'extraction de caractéristiques par Analyse en Composantes Principales (ACP), la maîtrise des





coûts est une contrainte stricte du cahier des charges.

- **Ressources éphémères** : Le cluster de préparation des données (Data Engineering) doit être provisionné à la demande et détruit de manière automatisée dès la fin de la génération des datasets. Il ne doit y avoir aucune facturation de ressources de calcul inactives.
- **Instances Spot** : L'architecture doit prioriser l'utilisation de la capacité excédentaire d'AWS (*Instances Spot*) pour les tâches de traitement par lots, permettant de réduire les coûts de calcul de manière significative (jusqu'à 70% d'économie).

2.2.3.3 Axe 3 : Sécurité, Isolement et Conformité

La manipulation de données propriétaires (images de cultures, modèles d'entreprise) nécessite un cloisonnement strict du système, conformément aux bonnes pratiques Cloud.

- **Isolation réseau (VPC)** : L'ensemble des ressources (S3, EMR, SageMaker Studio) doit être déployé au sein d'un Réseau Privé Virtuel (*Amazon VPC*), empêchant tout accès direct depuis l'Internet public.
- **Gestion des accès (IAM)** : Application stricte du « principe du moindre privilège ». Chaque service AWS (ex : le cluster EMR) et chaque pipeline d'automatisation (*GitHub Actions*) ne doit disposer que des autorisations strictement nécessaires à son exécution.
- **Chiffrement** : Toutes les données stockées dans le Data Lake ainsi que les artefacts de modèles doivent être chiffrés au repos et en transit via le service AWS *KMS* (Key Management Service).

2.2.3.4 Axe 4 : Maintenabilité et Supervision (SRE)

L'architecture, bien que complexe sous le capot, doit rester observable et facilement maintenable par les équipes techniques.

- **Supervision centralisée** : Tous les journaux d'exécution (*logs*) d'EMR et les métriques de performance des modèles SageMaker doivent être centralisés dans *Amazon CloudWatch* pour faciliter le débogage.
- **Reproductibilité (IaC)** : Toute modification de l'infrastructure doit pouvoir être reproduite et auditée. L'utilisation de *Terraform* garantit que l'environnement est versionné et auditable comme du code classique.





TABLEAU 2.3 – Matrice de synthèse des besoins non fonctionnels de l'architecture

Catégorie	Exigence technique	Solutions & Services AWS
Élasticité	Redimensionnement automatique selon la charge de données	<i>Amazon EMR / SageMaker Auto Scaling</i>
FinOps	Utilisation de clusters éphémères et d'instances à coût réduit	<i>Instances Amazon EC2 Spot</i>
Sécurité	Chiffrement des données, moindre privilège et isolement réseau	<i>AWS KMS, AWS IAM, Amazon VPC</i>
Supervision	Traçabilité centralisée des métriques et des erreurs	<i>Amazon CloudWatch</i>
Maintenabilité	Infrastructure déployable par code, reproductible et versionnée	<i>Terraform, GitHub Actions</i>

2.2.4 Acteurs du système

L'industrialisation d'un modèle de vision par ordinateur via le MLOps est un effort pluridisciplinaire. Le système conçu implique l'interaction de plusieurs acteurs aux profils techniques distincts. Nous les avons répartis en deux grandes catégories : l'équipe Data (qui manipule la donnée et le modèle) et l'équipe Infrastructure (qui gère les ressources Cloud).





TABLEAU 2.4 – Cartographie des acteurs et de leurs responsabilités dans le système

Acteur	Rôle et interactions avec le système
Équipe Data	
Data Engineer (Ingénieur Données)	Responsable de la première partie de l'architecture. Il développe et exécute les scripts de traitement distribué (PySpark) sur le cluster <i>Amazon EMR</i> . Il s'assure que les images sont correctement ingérées, nettoyées, transformées (ACP), et sauvegardées de manière structurée dans le Data Lake (<i>Amazon S3</i>).
Data Scientist (Scientifique des Données)	Utilisateur principal de la plateforme MLOps. Depuis son environnement <i>Amazon SageMaker Studio</i> , il consomme les données propres préparées par le Data Engineer. Il conçoit le code du modèle, lance les tâches d'entraînement (<i>Training Jobs</i>) et enregistre les versions candidates du modèle dans le registre (<i>Model Registry</i>).
Lead Data (Responsable Data)	Superviseur de la qualité algorithmique. Il analyse les métriques de performance des modèles enregistrés par les Data Scientists. C'est lui qui déclenche manuellement ou valide l'approbation d'un modèle dans le registre, autorisant ainsi son passage vers l'environnement de production.
Équipe Infrastructure	
Administrateur Cloud (Ingénieur MLOps / Cloud)	Garant de la stabilité, de la sécurité et des coûts (FinOps). Il déploie l'infrastructure sous-jacente en tant que code (<i>Terraform</i>) via les pipelines CI/CD (<i>GitHub Actions</i>). Il configure le réseau (VPC), gère les permissions d'accès (IAM) et supervise l'état des serveurs et les journaux d'erreurs via <i>Amazon CloudWatch</i> .

2.2.5 Diagramme de cas d'utilisation

La modélisation UML (Unified Modeling Language) permet de cartographier formellement les interactions fonctionnelles entre ces acteurs et le système Cloud. Le diagramme de cas d'utilisation présenté ci-dessous illustre le périmètre de notre architecture hybride, en mettant en évidence la séparation des responsabilités.

Ce diagramme s'articule autour des macro-processus suivants :

- **Gestion des données massives** : Ingestion des images brutes et déclenchement du pré-traitement distribué (réservé au Data Engineer).
- **Cycle de vie du modèle** : Développement, entraînement et enregistrement du modèle (réservé au Data Scientist).
- **Gouvernance et Déploiement** : Approbation du modèle (Lead Data) et orchestration de l'infrastructure globale (Administrateur Cloud).



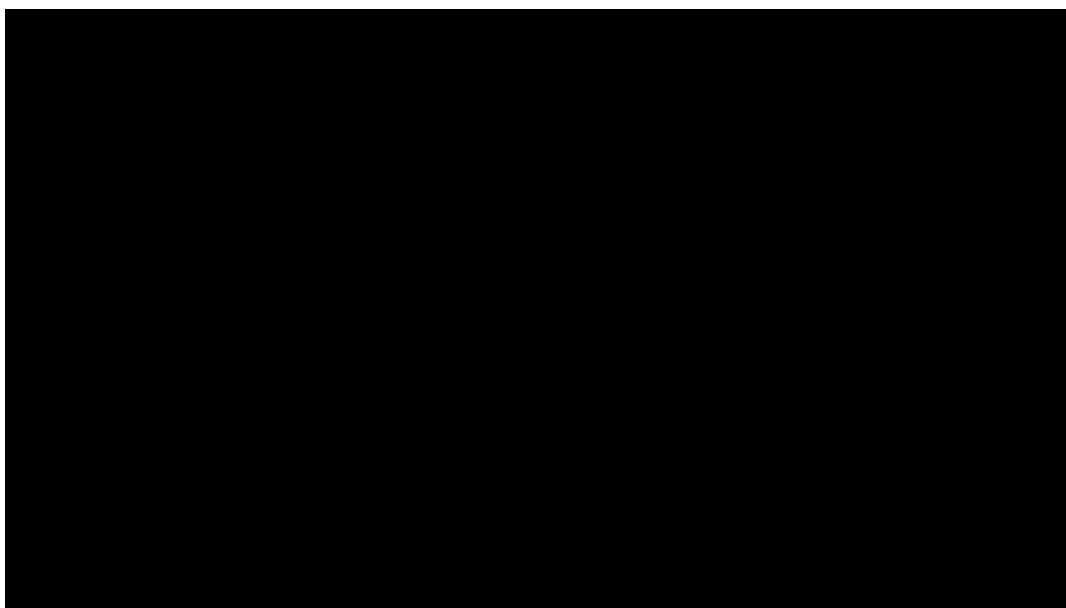


FIGURE 2.3 – Diagramme de cas d'utilisation du système hybride (Data Engineering & MLOps)

2.3 Conception de la solution

Sur la base de l'analyse des besoins fonctionnels et non fonctionnels, nous avons conçu une architecture Cloud distribuée capable de résoudre les problèmes de saturation locale rencontrés par le client. Le principe fondamental qui dicte cette conception est le **découplage strict entre le stockage des données et la puissance de calcul**, une bonne pratique incontournable dans l'ingénierie des données massives.

2.3.1 Architecture logique

L'architecture logique modélise le flux de traitement de l'information et l'orchestration des processus, en faisant abstraction des composants matériels ou des serveurs physiques sous-jacents. Ce modèle s'articule autour d'un pipeline divisé en deux grands sous-systèmes interdépendants, conformément à notre positionnement : le sous-système de préparation des données (*Data Engineering*) et le sous-système d'orchestration des modèles (*MLOps*).

Le flux logique de notre système suit un graphe orienté qui se décompose en quatre étapes majeures :

1. **Ingestion et Stockage centralisé** : L'opérateur métier ou le système de collecte dépose un jeu d'images agricoles brutes dans le référentiel central (*Data Lake*). Cet espace agit comme l'unique source de vérité (*Single Source of Truth*) pour l'ensemble du système, isolant les données brutes des données transformées.





2. **Prétraitement distribué (Pipeline Data) :** Le dépôt des données justifie l'instanciation du moteur de calcul distribué. Ce dernier parallélise les tâches lourdes de préparation : nettoyage des images, redimensionnement, et surtout la réduction de dimensionnalité via une Analyse en Composantes Principales (ACP). Les caractéristiques extraites (*features*) sont re-sauvegardées de manière structurée dans le Data Lake, allégeant ainsi le volume d'information.
3. **Orchestration et Entraînement (Pipeline ML) :** Une fois les données préparées, le moteur d'orchestration MLOps prend le relais. Il charge les données propres et exécute les scripts d'apprentissage du modèle de vision par ordinateur. Cette étape inclut la validation des performances du modèle candidat sur un jeu de test.
4. **Gouvernance et Inférence :** Le modèle entraîné est versionné et stocké dans un registre logique (*Model Registry*). Après validation (approbation) par l'équipe Data, il est déployé sur un point de terminaison (*Endpoint*) pour réaliser des prédictions (inférence) sur de nouvelles images. Les résultats prédictifs (détection d'anomalies) sont ensuite sauvegardés dans le Data Lake, clôturant ainsi le cycle logique.

Cette séparation logique garantit que la complexité et la lourdeur du traitement de la donnée brute n'interfèrent pas avec la plateforme de modélisation, assurant ainsi la fluidité et la répétabilité du cycle de vie du modèle.

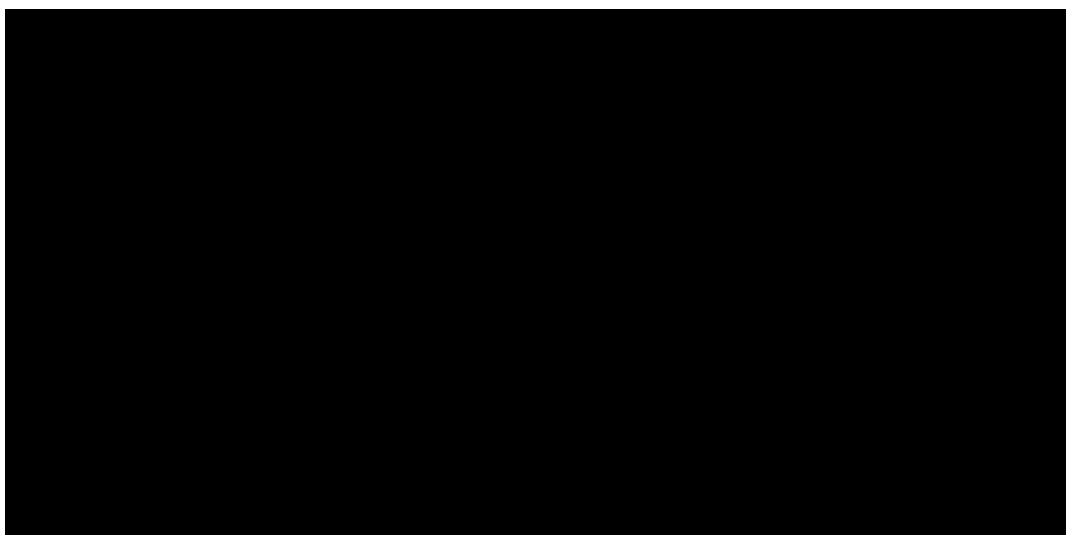


FIGURE 2.4 – Architecture logique du pipeline de traitement distribué et d'orchestration MLOps

2.3.2 Architecture physique

L'architecture physique matérialise le flux logique décrit précédemment en s'appuyant exclusivement sur les services Cloud managés d'Amazon Web Services (AWS). Conformément à notre positionnement hybride, cette infrastructure déploie des ressources distinctes pour le traitement





massif d'un côté, et pour l'orchestration du Machine Learning de l'autre, tout en garantissant un environnement hautement sécurisé.

Les composants physiques de notre architecture se répartissent en quatre couches fonctionnelles :

2.3.2.1 Couche 1 : Sécurité et Réseau (Fondations)

Toute l'architecture repose sur une fondation sécurisée pour protéger les données propriétaires du client :

- **Amazon VPC (Virtual Private Cloud)** : L'ensemble des ressources de calcul et des espaces de travail est isolé dans un réseau privé virtuel. Cela permet de contrôler strictement le trafic entrant et sortant via des groupes de sécurité (*Security Groups*) [6].
- **AWS IAM (Identity and Access Management)** : La gouvernance des accès est dictée par des rôles IAM stricts. Par exemple, le cluster EMR ne possède que les droits de lecture/écriture sur le bucket S3 dédié, garantissant l'application du principe de moindre privilège [6].

2.3.2.2 Couche 2 : Sous-système Data Engineering (Partie 1)

Cette couche absorbe la charge lourde du traitement des images agricoles :

- **Amazon S3 (Data Lake)** : Service de stockage objet agissant comme le cœur de l'architecture. Il stocke les images brutes, les données transformées (après ACP) et les artefacts des modèles de manière hautement durable et chiffrée.
- **Amazon EMR (Elastic MapReduce)** : Le moteur de calcul distribué (utilisant *Apache Spark / PySpark*). Pour respecter nos contraintes FinOps, ce cluster est **éphémère** : il est instancié avec des instances *Spot* (à coût réduit) uniquement pour la durée de la génération des datasets, puis détruit automatiquement.

2.3.2.3 Couche 3 : Plateforme MLOps (Partie 2)

Une fois les jeux de données préparés sur S3, cette couche prend le relais pour le cycle de vie de l'IA :

- **Amazon SageMaker Studio** : L'environnement de développement (*IDE*) managé fourni aux Data Scientists. Il permet de développer le code ML dans des Notebooks Jupyter connectés aux données de S3.
- **SageMaker Model Registry** : Un registre physique centralisant les versions des modèles entraînés et gérant leur statut d'approbation (ex : *PendingManualApproval* vers *Approved*) [6].





- **SageMaker Endpoints** : L'infrastructure d'inférence. Les modèles validés y sont déployés sous forme d'API REST auto-scalables pour réaliser des prédictions sur de nouvelles images.

2.3.2.4 Couche 4 : Automatisation et Supervision (CI/CD et IaC)

Pour relier les composants et assurer la reproductibilité de l'infrastructure :

- **Terraform & GitHub Actions** : L'infrastructure AWS et les pipelines MLOps ne sont pas cliqués manuellement, mais définis sous forme de code (*Infrastructure as Code*). GitHub Actions orchestre le déploiement continu des ressources via Terraform, garantissant que chaque environnement peut être recréé à l'identique.
- **Amazon CloudWatch** : Service de supervision qui collecte les métriques d'utilisation (CPU, RAM) des clusters EMR et les journaux d'erreurs des modèles SageMaker pour alerter l'Administrateur Cloud en cas de défaillance.

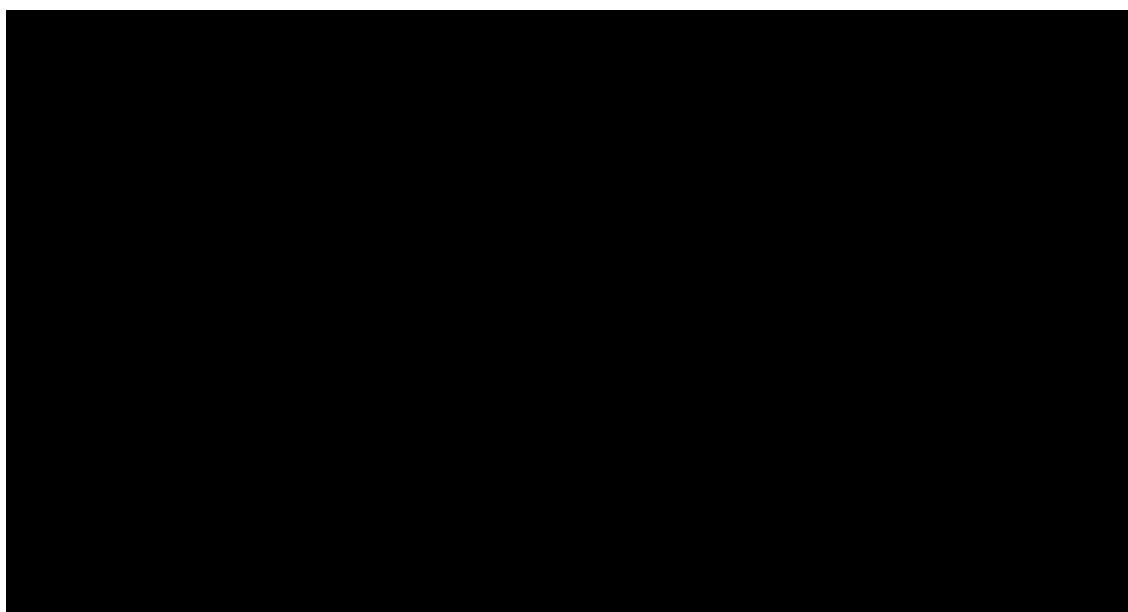


FIGURE 2.5 – Architecture physique détaillée de la solution MLOps sur Amazon Web Services

2.4 Résumé du chapitre

Ce chapitre a permis de traduire la problématique d'ingénierie soulevée par le client en une solution technique concrète et rigoureusement modélisée. En appliquant une méthodologie de conception itérative basée sur des Sprints, nous avons identifié les limites critiques de l'approche locale initiale et formalisé des besoins stricts en matière de passage à l'échelle, de sécurité et d'optimisation financière (FinOps).





L'analyse des acteurs a clarifié la répartition des tâches au sein de l'organisation : le *Data Engineer* et le *Data Scientist* interagissant avec les données et les modèles, tandis que le *Lead Data* et l'*Administrateur Cloud* assurent la gouvernance et le maintien de l'infrastructure.

Enfin, la conception architecturale a mis en évidence notre choix stratégique fondamental : un découplage physique via une architecture hybride. Le traitement massif des données est intégralement délégué à un sous-système Big Data éphémère (Amazon EMR et S3), tandis que l'orchestration, le registre et le déploiement du modèle d'intelligence artificielle reposent sur une plateforme MLOps standardisée (Amazon SageMaker, couplé à GitHub et Terraform).

Ces fondations méthodologiques et architecturales étant désormais solidement établies, le chapitre suivant sera consacré à l'implémentation pratique de cette architecture sur le Cloud AWS et à l'évaluation de ses performances.



IMPLÉMENTATION ET RÉSULTATS

Le chapitre précédent a défini la modélisation fonctionnelle et physique de notre solution MLOps. Ce dernier chapitre est consacré à sa mise en œuvre concrète sur Amazon Web Services (AWS) ainsi qu'à l'évaluation technique et financière de la solution. Il présente l'implémentation de l'infrastructure matérielle (Cloud) et logicielle, le développement des modules, ainsi que les résultats expérimentaux obtenus.

Sommaire

3.1	Implémentation matérielle	39
3.1.1	Configuration de l'environnement Cloud	39
3.1.2	Ressources d'infrastructure nécessaires (Services Cloud)	39
3.1.3	Bilan FinOps et Consommation des ressources	42
3.2	Implémentation logicielle	43
3.2.1	Outils logiciels et environnement de travail collaboratif	43
3.2.2	Choix des technologies	44
3.2.3	Développement des modules	44
3.3	Résultats	45
3.3.1	Intégration et tests	45
3.3.2	Protocole Expérimental	45
3.3.3	Résultats Expérimentaux	46
3.4	Discussion	46
3.5	Résumé du chapitre	47



3.1 Implémentation matérielle

Nous présentons ici la plateforme d'infrastructure chargée d'exécuter notre système de traitement. Contrairement à une approche physique locale classique, notre "matériel" est entièrement virtualisé sur le Cloud Amazon Web Services (AWS) et configuré pour le calcul distribué.

3.1.1 Configuration de l'environnement Cloud

La plateforme Cloud chargée d'exécuter notre système MLOps est structurée autour du découplage strict entre le stockage, le réseau et le calcul. Le tableau 3.1 synthétise la configuration matérielle et logicielle mobilisée pour ces expérimentations.

TABLEAU 3.1 – Environnement matériel et logiciel de l'implémentation

Élément	Configuration retenue
Cloud Provider	Amazon Web Services (AWS)
Isolation	Virtual Private Cloud (VPC)
Sécurité	Identity and Access Management (IAM)
Stockage (Data Lake)	Amazon S3
Calcul distribué	Amazon EMR
Framework Big Data	Apache Spark
Langage & Environnement	Python / PySpark / Jupyter Notebook
Système d'exploitation	Linux
Monitoring	Cloud Watch

3.1.2 Ressources d'infrastructure nécessaires (Services Cloud)

3.1.2.1 Réseau sécurisé : Virtual Private Cloud (VPC)

Conformément à notre approche de sécurité, le déploiement a débuté par la création d'un VPC. Ce service permet d'isoler le projet dans un réseau virtuel défini logiquement, garantissant que les clusters et les espaces de travail ne soient accessibles que par des canaux sécurisés et autorisés.



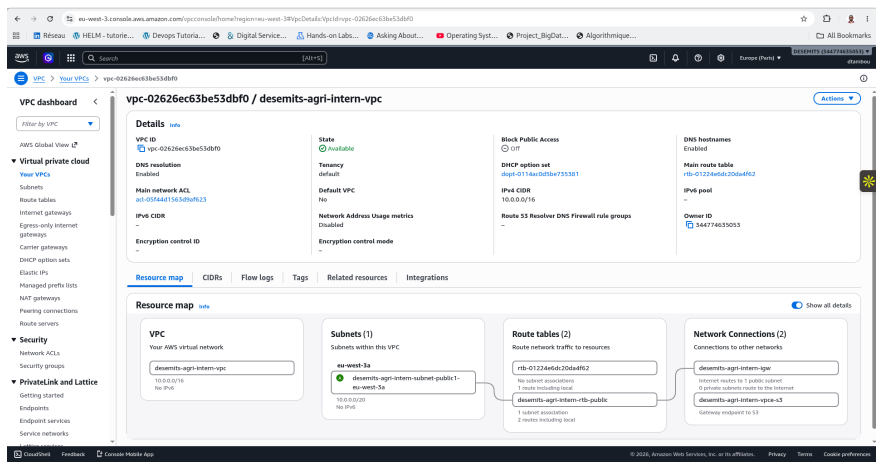


FIGURE 3.1 – Virtual Private Cloud (VPC) créé pour l’isolation du projet

3.1.2.2 L’espace de stockage : Amazon S3 (Data Lake)

Le composant matériel de stockage de notre architecture est un compartiment (*bucket*) Amazon S3 faisant office de Data Lake. Afin d’assurer une traçabilité stricte du cycle de vie de la donnée, ce stockage a été structuré en plusieurs répertoires distincts, séparant hermétiquement les images brutes des résultats finaux et des journaux d’exécution.

TABLEAU 3.2 – Organisation du Data Lake sur Amazon S3

Répertoire	Contenu et rôle
PCA/	Contient les données transformées après réduction de dimension par Analyse en Composantes Principales (ACP).
Results_features/	Stocke les caractéristiques extraites ainsi que les résultats intermédiaires et finaux.
Test/	Jeu de données de test utilisé pour l’évaluation et la validation du modèle.
Test-grand/	Version étendue du jeu de test pour les expérimentations à plus grande échelle.
models/	Contient les modèles entraînés ainsi que les fichiers nécessaires à leur déploiement.
error-logs/	Regroupe les journaux d’exécution générés par Amazon EMR pour assurer le débogage.
studios/	Répertoire utilisé par Amazon EMR Studio pour le stockage des espaces de travail.
bootstrap-emr.sh	Script d’initialisation exécuté lors du démarrage du cluster EMR.



```

/mnt/dtamboudisk/rapport_ing_aia$ aws s3 ls s3://desemits-s3-agri-intern-dt/
PRE PCA/
PRE Results_features/
PRE Test-grand/
PRE Test/
PRE error-logs/
PRE models/
PRE studios/
215 bootstrap-emr.sh

```

FIGURE 3.2 – Organisation du Data Lake dans le bucket Amazon S3

3.1.2.3 Le moteur de calcul : Amazon EMR

Une fois les données sécurisées, un cluster Amazon EMR a été déployé pour exécuter les calculs. Il est composé d'un nœud maître (orchestrateur) et de plusieurs nœuds de travail (*workers*) chargés de paralléliser les tâches. Une connexion sécurisée au nœud maître peut être établie via le protocole SSH pour permettre à l'administrateur d'effectuer des opérations de maintenance et d'analyse d'incidents.

The screenshot displays the Amazon EMR console for a cluster named 'desemits-emr-cluster-agri-intern-4'. The cluster is in a 'Waiting' status. Key details include:

- Cluster ID:** j-1E7PCJ7F9YFR7
- Cluster ARN:** arn:aws:elasticmapreduce:eu-west-3:344774635053:cluster/j-1E7PCJ7F9YFR7
- Capacity:** 1 Primary, 4 Core, 0 Task
- Applications:** Amazon EMR version emr-7.13.0. Installed applications include AmazonCloudWatchAgent 1.300052.2, Hadoop 3.4.2, Hive 3.13.3, JupyterEnterpriseGateway 2.6.0, Uby 0.8.0, Spark 3.5.6, and TensorFlow 2.19.0.
- Cluster management:** Log destination in Amazon S3 (desemits-s3-agri-intern-dt/error-logs) and Amazon CloudWatch (aws/emr/j-1E7PCJ7F9YFR7). Persistent application UIs include Spark History Server, YARN Timeline Server, and Tez UI.
- Status and time:** Status is 'Waiting'. Creation time is July 13, 2026, 03:43 (UTC+01:00). Elapsed time is 36 minutes, 15 seconds.
- Operating system:** Amazon Linux release 2023.12.20260622.0.
- Cluster logs:** Archive log files to Amazon S3 is turned on. Amazon S3 location is s3://desemits-s3-agri-intern-dt/error-logs/.
- Cluster termination and node replacement:** Termination option is 'Automatically terminate cluster after idle time'. Idle time is 2 hours, 50 minutes. Termination protection is 'Off'. Unhealthy node replacement is 'On'.

FIGURE 3.3 – Déploiement du cluster Amazon EMR (1 Nœud maître et 4 Workers)



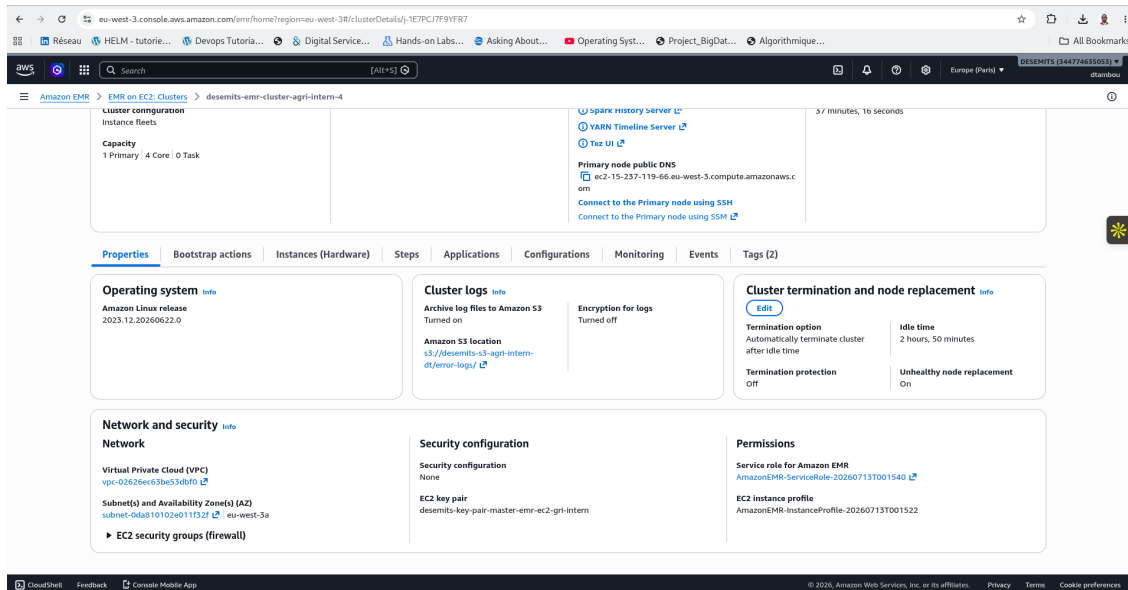


FIGURE 3.4 – Déploiement du cluster Amazon EMR (1 Nœud maître et 4 Workers) : Fin

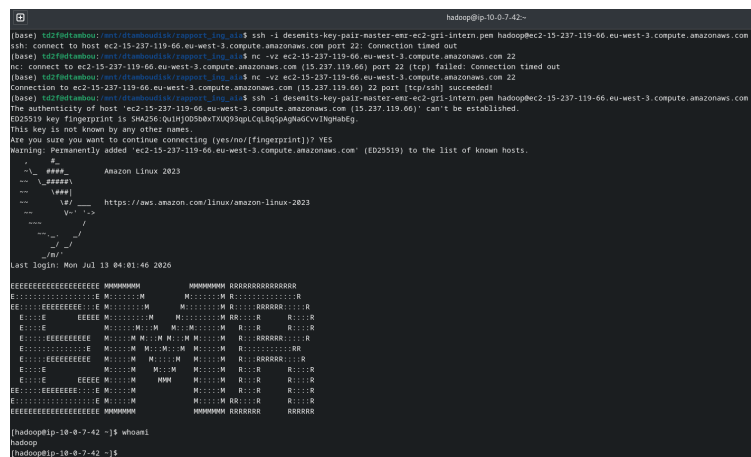


FIGURE 3.5 – Connexion sécurisée au nœud maître Amazon EMR via SSH

3.1.3 Bilan FinOps et Consommation des ressources

3.1.3.1 Consommation des ressources matérielles virtuelles

Dans une architecture Cloud, la consommation ne se mesure pas en Watts mais en temps d'allocation de serveurs. Pour répondre à l'exigence majeure d'optimisation (FinOps), notre cluster EMR a été configuré pour être strictement **éphémère**. Il se lance au clonage d'un modèle existant et se détruit automatiquement après un temps défini (généralement 2h), évitant ainsi la consommation de ressources inactives.





3.1.3.2 Calcul et optimisation des coûts

AWS fournit des outils de suivi permettant d'analyser précisément les dépenses engendrées par chaque service utilisé. Les principaux coûts proviennent de l'exécution du cluster Amazon EMR et des instances EC2 associées. Grâce à l'extinction automatique du cluster, le budget alloué a été rigoureusement respecté pour chaque expérimentation.

3.2 Implémentation logicielle

L'implémentation de la partie logicielle de notre système a été réalisée en plusieurs étapes : le choix des outils, la configuration de l'environnement de travail collaboratif, et le développement des scripts de traitement distribué.

3.2.1 Outils logiciels et environnement de travail collaboratif

3.2.1.1 Outils logiciels (Environnement de l'opérateur métier)

L'un des enjeux logiciels majeurs était de fournir un environnement de développement interactif, sans forcer l'utilisateur à administrer le matériel sous-jacent.

Amazon EMR Studio : Il s'agit de l'interface web de développement proposée par AWS pour les traitements Big Data. Il centralise l'accès aux notebooks Jupyter et aux clusters EMR, permettant à l'utilisateur de se concentrer exclusivement sur ses données.

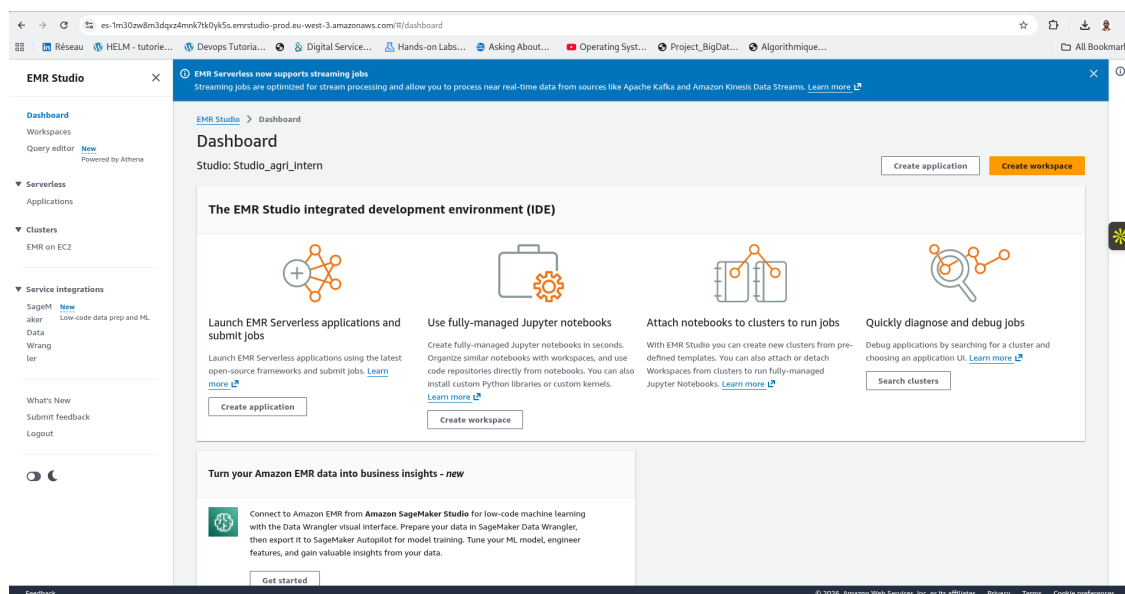


FIGURE 3.6 – Interface d'Amazon EMR Studio

Workspace Collaboratif (Jupyter) : Au sein d'Amazon EMR Studio, les traitements sont réalisés dans un *Workspace*. Le Workspace est totalement indépendant des ressources de calcul.



Un cluster Amazon EMR peut y être attaché ou détaché selon les besoins. Cette approche permet au Data Scientist de développer ses notebooks de manière transparente, sans se préoccuper du cycle de vie du cluster éphémère.

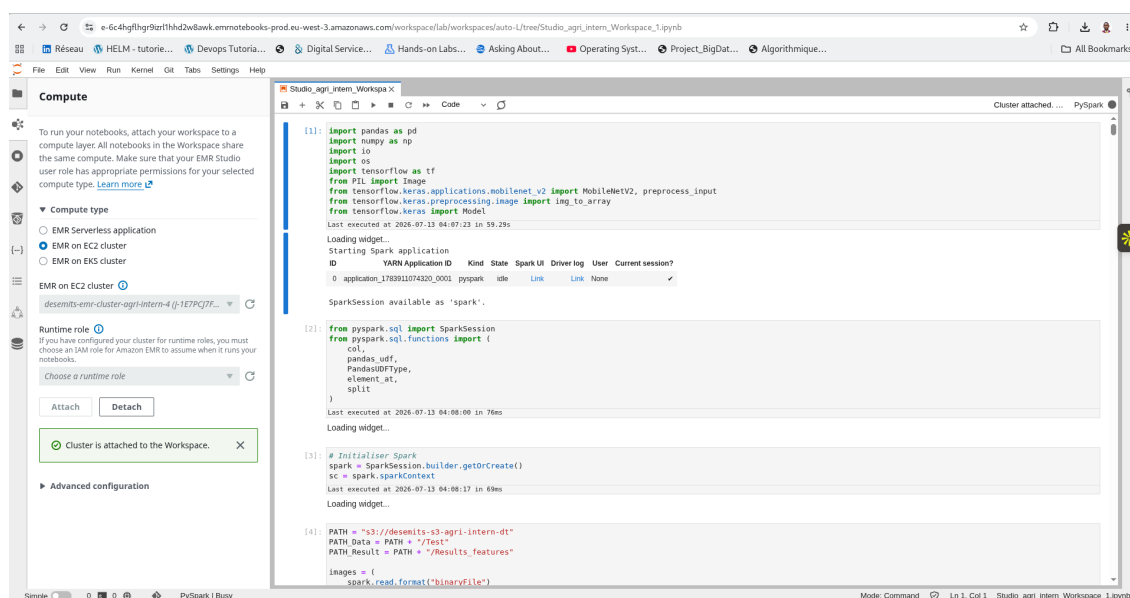


FIGURE 3.7 – Workspace collaboratif associé à un cluster Amazon EMR

3.2.2 Choix des technologies

3.2.2.1 Langages de programmation et Framework

Python : Langage principal utilisé pour l'ensemble des scripts de modélisation au sein de l'espace de travail. Il bénéficie d'une vaste bibliothèque facilitant la création et l'analyse de données.

PySpark : Il s'agit de l'API Python pour Apache Spark, le moteur de calcul distribué. PySpark a été utilisé pour écrire le pipeline de traitement des données, distribuer l'Analyse en Composantes Principales (ACP) et paralléliser l'ingestion depuis le Data Lake S3.

3.2.3 Développement des modules

Le développement logiciel s'est concentré sur la création d'un pipeline PySpark unifié. Ce script est exécuté depuis le Workspace Jupyter et se charge de coordonner :

- L'ingestion distribuée des images depuis Amazon S3.
- Le prétraitement et la réduction de dimensionnalité via ACP en exploitant la RAM des nœuds de travail.
- La sauvegarde automatisée des résultats finaux dans les sous-répertoires du Data Lake étudiés précédemment.



3.3 Résultats

Cette section présente les résultats obtenus lors des phases de test et de validation de notre système distribué sur AWS.

3.3.1 Intégration et tests

Les expérimentations menées ont permis de valider le bon fonctionnement de l'infrastructure Cloud distribuée. Après l'intégration du cluster EMR avec Amazon S3, les traitements ont été exécutés avec succès.

- **Test de supervision via l'interface Spark :** L'interface Web native d'Apache Spark a permis de suivre en temps réel l'exécution des différents traitements. L'interface décompose chaque *Job* en un ensemble de *Stages*, eux-mêmes constitués de plusieurs tâches (*Tasks*) exécutées en parallèle sur les différents nœuds du cluster.

Job Id (Job Group)	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0 (4)	Listing leaf files and directories for 131 paths: s3://desemits-s3-agi-intern-dt/Test/Apple Braeburn, ...	2026/07/13 03:08:42	10 s	1/1	131/131
1 (4)	Job group for statement 4	2026/07/13 03:08:54	1 s	1/1	1/1
2 (11)	Job group for statement 11	2026/07/13 03:09:54	9.9 min	1/1	709/709
3 (11)	Job group for statement 11	2026/07/13 03:19:48	7.6 min	1/1 (1 skipped)	24/24 (709 skipped)
4 (13)	Job group for statement 13	2026/07/13 03:28:06	0.2 s	1/1	1/1
5 (13)	Job group for statement 13	2026/07/13 03:28:06	0.5 s	1/1	1/1
6 (13)	Job group for statement 13	2026/07/13 03:28:07	2 s	1/1	1/1
7 (13)	Job group for statement 13	2026/07/13 03:28:08	1.0 s	1/1	1/1
8 (13)	Job group for statement 13	2026/07/13 03:28:09	11 s	1/1	3/3
9 (13)	Job group for statement 13	2026/07/13 03:28:20	0.3 s	1/1	1/1
10 (13)	Job group for statement 13	2026/07/13 03:28:21	15 s	1/1	3/3

FIGURE 3.8 – Interface Apache Spark présentant la répartition des tâches sur le cluster

3.3.2 Protocole Expérimental

Pour évaluer les performances globales du système, un protocole expérimental a été mis en place, comprenant les éléments suivants :

3.3.2.1 Environnement expérimental

Un cluster Amazon EMR instancié de manière éphémère et connecté à l'espace de travail Jupyter de l'opérateur métier. Les données sources (images) sont placées dans le répertoire S3 défini à cet effet.



3.3.2.2 Scénarios de test

Le scénario principal évalué est : « L'exécution complète du pipeline distribué PySpark sur un jeu de données volumineux, depuis le prétraitement jusqu'à l'enregistrement des résultats finaux, suivi de la libération des ressources. »

3.3.2.3 Métriques de performance

- **Réussite de l'intégration** : Validation du dialogue entre le Data Lake, le Workspace et les nœuds de calcul.
- **Optimisation FinOps** : Validation du respect des contraintes financières par l'utilisation de clusters éphémères.

3.3.3 Résultats Expérimentaux

Le Tableau 3.3 synthétise le bilan des résultats obtenus pour chaque critère.

TABLEAU 3.3 – Bilan de l'évaluation de l'infrastructure Cloud

Critère évalué	Statut	Observation
Déploiement du cluster EMR	Validé	Création et configuration réussies du cluster distribué Amazon EMR.
Stockage distribué	Validé	Centralisation des données et des résultats dans le Data Lake Amazon S3.
Exécution des traitements Spark	Validée	Les traitements distribués ont été exécutés et supervisés via l'interface Apache Spark sans saturation mémoire.
Optimisation FinOps	Validée	Utilisation d'un cluster éphémère limitant la consommation des ressources Cloud aux seules périodes d'exécution.

3.4 Discussion

Les objectifs fixés dans le cadre de ce projet ont été atteints en démontrant la faisabilité et l'efficacité d'une architecture distribuée reposant sur Amazon S3, Amazon EMR et Apache Spark. En palliant les problèmes de saturation liés à l'approche locale de l'alternant précédent, l'opérateur métier dispose désormais d'un environnement (Workspace) hautement performant.

Néanmoins, l'analyse critique des résultats révèle plusieurs pistes d'amélioration qui pourraient être envisagées pour renforcer le niveau d'automatisation de la plateforme.





3.5 Résumé du chapitre

Ce chapitre a présenté l'implémentation complète de l'architecture Cloud MLOps sur AWS, depuis le provisionnement de la partie matérielle virtuelle (VPC, S3, EMR) jusqu'à la mise en place de la partie logicielle (Workspaces Jupyter et PySpark). Les résultats expérimentaux confirment la faisabilité, la scalabilité et la viabilité financière de notre système pour le traitement massif d'images agricoles. La section suivante présentera la conclusion générale du mémoire et clôturera ce travail.



CONCLUSION GÉNÉRALE

Cette dernière section résume les principales contributions du projet, met en évidence ses limites et décrit les perspectives futures et un plan d'amélioration à partir du système existant.

Rappel du problème

Le présent travail s'est concentré sur la résolution des problèmes de saturation matérielle rencontrés par le client de DESEMITs. L'enjeu principal était d'industrialiser un modèle expérimental de vision par ordinateur, dont l'exploitation en production était freinée par les limites de calcul et de mémoire d'une exécution locale monolithique, incapable de traiter l'augmentation massive des images agricoles.

Solution : Démarche, résultats et intérêt

Pour surmonter ces obstacles, notre démarche s'est appuyée sur les principes du *Machine Learning Operations* (MLOps) et du calcul distribué. L'analyse des travaux existants a guidé la conception d'une architecture Cloud sur-mesure, dissociant strictement le stockage massif (Amazon S3) de la puissance de calcul (Amazon EMR). L'implémentation de cette solution à l'aide d'Apache Spark a permis de paralléliser efficacement le prétraitement par réduction de dimension (ACP) et l'inférence du réseau de neurones convolutifs.

Les résultats obtenus valident notre hypothèse de départ : le couplage de services Cloud managés et de pipelines distribués permet non seulement d'assurer le passage à l'échelle du système, mais également de maîtriser drastiquement les coûts opérationnels (FinOps) grâce à l'allocation éphémère des clusters.

Limites

Bien que les résultats obtenus posent les fondations robustes d'un système intelligent et hautement disponible, ce travail présente certaines limites. Tout d'abord, l'architecture nécessite encore d'être éprouvée sur des volumétries de données plus extrêmes. Par la suite, le déploiement manuel de l'infrastructure via la console par l'administrateur, ainsi que la nécessité pour l'opérateur métier d'exécuter manuellement ses scripts, constituent un frein à l'automatisation



complète du cycle de vie du modèle. Ces limites opérationnelles sont autant de défis qui restent à relever.

Perspectives d'amélioration

Pour pallier ces limites et poursuivre l'industrialisation du système, les perspectives futures incluent l'intégration de nouveaux outils visant l'automatisation totale :

- **Automatisation de l'infrastructure** : Le déploiement du cluster, aujourd'hui manuel, pourrait être entièrement scripté via l'Infrastructure as Code (IaC), permettant un déclenchement automatique dès l'arrivée de nouvelles images sur S3.
- **Orchestration des Workflows** : L'intégration d'un outil comme **Apache Airflow** permettrait d'enchaîner automatiquement les différentes étapes du pipeline sans exécution manuelle des scripts Jupyter.
- **Gouvernance MLOps complète** : L'ajout d'outils tels que **MLflow** ou **Amazon SageMaker Pipelines** constituerait une évolution naturelle pour automatiser le réentraînement des modèles (Continuous Training) et assurer un contrôle strict du versionnage des algorithmes au fil des saisons agricoles.

En ouvrant la voie à l'intégration de ces outils de surveillance et de réentraînement continu, ce travail démontre que l'avenir de l'intelligence artificielle ne réside plus uniquement dans l'optimisation des algorithmes, mais dans l'excellence de l'ingénierie logicielle et Cloud qui les supporte.



RÉFÉRENCES BIBLIOGRAPHIQUES

- [1] APACHE SOFTWARE FOUNDATION. *Apache Spark™ - Unified Engine for large-scale data analytics*. Visité le 10/03/2026. 2026. URL : <https://spark.apache.org/>.
- [2] AMAZON WEB SERVICES. *New Whitepaper: Amazon Elastic MapReduce (EMR) Best Practices*. Visité le 31/05/2026. 2025. URL : <https://aws.amazon.com/blogs/aws/>.
- [3] Jack YE et Yun ZOU. *Apache Polaris and Lance: Bringing AI-Native Storage to the Open Multimodal Lakehouse*. Visité le 07/03/2026. 2026. URL : <https://polaris.apache.org/blog/2026/01/06/apache-polaris-and-lance-bringing-ai-native-storage-to-the-open-multimodal-lakehouse/>.
- [4] AMAZON WEB SERVICES. *Best practices to optimize data access performance from Amazon EMR and AWS Glue to Amazon S3*. Visité le 28/03/2026. 2025. URL : <https://aws.amazon.com/blogs/big-data/>.
- [5] Rachana PAWAR et Dinesh RIVANKAR. *LLMOps: A Strategic Framework for Effective AI Lifecycle Management*. Visité le 04/05/2026. 2025. URL : <https://www.persistent.com/blogs/llmops-a-strategic-framework-for-effective-ai-lifecycle-management/>.
- [6] AMAZON WEB SERVICES. *Machine Learning Lens - AWS Well-Architected Framework*. Rapp. tech. AWS, 2025.
- [7] Nico RENALDO. “Cost-Efficient MLOps Architecture for Scientific Machine Learning”. Mém. de mast. Tampere University, 2025.
- [8] Syed JAFFRY et Saurabh TRIKANDÉ. *Model hosting patterns in Amazon SageMaker, Part 3: Run and optimize multi-model inference*. Visité le 24/05/2026. 2022. URL : <https://aws.amazon.com/blogs/machine-learning/>.
- [9] S. PRASAD et U. ARORA. “Scalable Inference Architectures for Compound AI Systems: A Production Deployment Study”. In : *arXiv* (2025).
- [10] Lavanya SHANMUGAM, Kumaran THIRUNAVUKKARASU, Jesu Narkarunai Arasu MALAIYAPPAN et Sanjeev PRAKASH. “Scalable Cloud Architectures for Distributed Machine Learning: A Comparative Analysis”. In : *International Journal for Multidisciplinary Research (IJFMR)* 6.1 (2024). ISSN : 2582-2160.



- [11] Various AUTHORS. “Microservice Architecture Patterns for Scalable Machine Learning Systems”. In : *arXiv* (2025).
- [12] Oleh PITSUN et Myroslav SHYMCHUK. “Scalable MLOps Pipeline with Complexity-Driven Model Selection Using Microservices”. In : *Technologies* 14.1 (2026), p. 45.
- [13] AMAZON WEB SERVICES. *Best practices and design patterns for building machine learning workflows with Amazon SageMaker Pipelines*. Visité le 25/03/2026. 2025. URL : <https://aws.amazon.com/blogs/machine-learning/>.
- [14] AMAZON WEB SERVICES. *Orchestrate end-to-end scalable ETL pipeline with Amazon SageMaker Workflows*. Visité le 08/06/2026. 2025. URL : <https://aws.amazon.com/sagemaker/>.
- [15] RESEARCHGATE. (PDF) *Unleashing PySpark + AI/GenAI: Scalable Data Processing for Modern Enterprises*. Article scientifique. 2026.
- [16] LEEWAYHERTZ. *How to build an MLOps pipeline?* Visité le 26/04/2026. 2025. URL : <https://www.leewayhertz.com/>.
- [17] APACHE SOFTWARE FOUNDATION. *ML Pipelines - Spark 4.1.2 Documentation*. Visité le 09/05/2026. 2025. URL : <https://spark.apache.org/docs/latest/ml-pipeline.html>.
- [18] WIKIPEDIA CONTRIBUTORS. *MLOps — Wikipédia, l'encyclopédie libre*. Visité le 11/05/2026. 2025. URL : <https://fr.wikipedia.org/wiki/MLOps>.
- [19] NEURIPS. *Machine Learning for Creativity and Design: Introduction*. Visité le 21/05/2026. 2020. URL : <https://neurips2020creativity.github.io/>.
- [20] Youssef SOUATI. “AwalTiraWhisper: An MLOps End-to-End Production-Ready Pipeline”. Capstone Design Final Report. Al Akhawayn University - School of Science & Engineering, 2025.
- [21] Dominik KREUZBERGER, Niklas KÜHL et Sebastian HIRSCHL. “Machine Learning Operations (MLOps): Overview, Definition, and Architecture”. In : *IEEE Access* 11 (2023).
- [22] Emily CARTER. *A Glossary of MLOps Terms Every CTO Should Understand in 2026*. Visité le 16/07/2026. 2026. URL : <https://hire-aidevelopers.com/blog/mlops-glossary-for-ctos-2026/>.
- [23] Christoph WINDHEUSER et Danilo SATO. *MLOps: Continuous Delivery for Machine Learning on AWS*. Rapp. tech. ThoughtWorks / Amazon Web Services, 2020.
- [24] D. SCULLEY et al. “Hidden Technical Debt in Machine Learning Systems”. In : *Advances in Neural Information Processing Systems (NIPS)*. 2015.





- [25] AWS EVENTS. *AWS re:Invent 2020: Designing better ML systems: Learnings from Netflix*. Vidéo YouTube, visité le 18/07/2026. 2020. URL : https://www.youtube.com/watch?v=netflix_reinvent.
- [26] AMAZON WEB SERVICES. *Riot Games Success Story - Scaling global game infrastructure using AWS Local Zones*. Visité le 25/07/2026. 2026. URL : <https://aws.amazon.com/solutions/case-studies/riot-games-local-zones-case-study/>.
- [27] Srikanth JONNAKUTI. "ENABLING AI-FIRST PRODUCT DESIGN: A SCALABLE CLOUD FOUNDATION FOR ML-DRIVEN INNOVATION". In : *International Journal of Engineering Technology Research & Management (IJETRM)* 07.06 (2023). ISSN : 2456-9348.
- [28] Rakib HOSSAIN et al. "Ethical and Explainable AI in Reusable MLOps Pipelines". In : *arXiv* (2024).
- [29] AMAZON WEB SERVICES. *Expedia Group on AWS: Case Studies, Videos, Innovator Stories*. Visité le 23/07/2026. 2026. URL : <https://aws.amazon.com/solutions/case-studies/innovators/expedia/>.
- [30] Jordan GRUBB et Irene Arroyo DELGADO. *Implement a secure MLOps platform based on Terraform and GitHub*. Visité le 24/07/2026. 2025. URL : <https://aws.amazon.com/blogs/machine-learning/implement-a-secure-mlops-platform-based-on-terraform-and-github/>.
- [31] AMAZON WEB SERVICES. *Best practices - AWS Prescriptive Guidance*. Visité le 22/03/2026. 2025. URL : <https://docs.aws.amazon.com/prescriptive-guidance/>.
- [32] AMAZON WEB SERVICES. *Netflix Case Study*. Visité le 25/07/2026. 2026. URL : <https://aws.amazon.com/solutions/case-studies/netflix-case-study/>.
- [33] AWS EVENTS. *AWS re:Invent 2024 - AI-driven value: Capital One's path to better customer experience (AIM130)*. Vidéo YouTube, visité le 20/07/2026. 2024. URL : https://www.youtube.com/watch?v=capitalone_reinvent.
- [34] CLOUD CONSULTING EUROPE GMBH (CCE). *Expedia case study - Above the clouds*. Visité le 21/07/2026. 2021. URL : <https://www.cloud-consulting.eu/>.
- [35] Romina SHARIFPOUR et Pooya VAHIDI. *Build an end-to-end MLOps pipeline using Amazon SageMaker Pipelines, GitHub, and GitHub Actions*. Visité le 22/07/2026. 2023. URL : <https://aws.amazon.com/blogs/machine-learning/build-an-end-to-end-mlops-pipeline-using-amazon-sagemaker-pipelines-github-and-github-actions/>.

